

TRANSFORMER

算法岗面试 100 题

题目驱动 • 深度推导 • Coding / Debug • 推理系统

原创面试手册 | 2026 Edition

参考《剑指 Offer》的“题目驱动 + 逐层分析 + 举一反三”学习节奏
但不复制其受版权保护的正文或版式。

适用：NLP / LLM / 搜索推荐 / 多模态算法岗 • 机器学习工程岗 • 大模型 Infra 交叉岗

使用说明：这本手册怎么刷

定位 不是“背 100 个答案”，而是训练从 公式 $\rightarrow$ Shape $\rightarrow$ 复杂度 $\rightarrow$ 数值稳定性 $\rightarrow$ 代码 $\rightarrow$ 系统的连续推导能力。

- 第一遍：只读“30 秒答案”，确保每题能在 30-60 秒内给出主干。
- 第二遍：手推公式与 shape，所有复杂度和 KV Cache 内存公式必须能脱稿写出。
- 第三遍：把 Coding 题真正实现，并用行为级单元测试验证，而不是只看代码能运行。
- 第四遍：随机抽题后沿“追问链”继续回答 3-5 层，训练高级面试的深挖能力。

难度标记

1-2/5 基础	3/5 高频主干	4/5 高级	5/5 核心/研究岗
----------	----------	--------	------------

知识主线：不要把 100 题看成 100 个孤岛

链 1: Attention 数学 点积方差 $\rightarrow \sqrt{d_k}$ $\rightarrow$ Softmax $\rightarrow$ Mask $\rightarrow$ Multi-Head $\rightarrow$ 复杂度

链 2: 位置 Permutation Equivariance $\rightarrow$ Position Encoding $\rightarrow$ Relative $\rightarrow$ RoPE $\rightarrow$ 长上下文外推

链 3: 深层训练 Residual $\rightarrow$ LayerNorm/RMSNorm $\rightarrow$ Pre-Norm $\rightarrow$ 初始化 $\rightarrow$ Warmup

链 4: MLP FFN $\rightarrow$ expansion ratio $\rightarrow$ GELU/SiLU $\rightarrow$ SwiGLU $\rightarrow$ MoE

链 5: 推理 Autoregressive $\rightarrow$ KV Cache $\rightarrow$ MQA/GQA $\rightarrow$ Paged KV $\rightarrow$ Continuous Batching

链 6: 系统 $O(T^2)$ 不等于实际慢点 $\rightarrow$ GPU IO $\rightarrow$ FlashAttention $\rightarrow$ quantization $\rightarrow$ serving SLA

全书目录与 100 题索引

第一章 Transformer 总体架构与设计动机 从 RNN 的瓶颈出发，建立 Encoder、Decoder、Residual Stream、Shape 与信息流的整体图景。

1. Q001 为什么 Transformer 能取代 RNN/LSTM?
2. Q002 请完整描述一个 Transformer Encoder Layer
3. Q003 请完整描述 Transformer Decoder
4. Q004 Residual Connection 到底解决什么问题?
5. Q005 为什么 Transformer 更常用 LayerNorm 而不是 BatchNorm?
6. Q006 手写 Transformer 时最重要的 Tensor Shape 链是什么?
7. Q007 标准 Multi-Head Attention 参数量如何估算?
8. Q008 Self-Attention 与 Cross-Attention 的本质区别是什么?

- 9. Q009 Attention 与 FFN 分别承担什么功能?
- 10. Q010 如果完全删除 Position Encoding, 会发生什么?

第二章 Attention 数学与实现细节 从点积方差、Softmax 数值稳定性、Mask 到复杂度, 要求能手推也能落地实现。

- 11. Q011 写出 Scaled Dot-Product Attention, 并逐项解释
- 12. Q012 为什么必须除以 $\sqrt{d_k}$?
- 13. Q013 为什么 Mask 要加在 Softmax 之前?
- 14. Q014 Softmax 如何避免数值溢出?
- 15. Q015 为什么需要 Multi-Head Attention?
- 16. Q016 为什么通常设置 $d_{head}=d_{model}/H$?
- 17. Q017 Q、K、V 为什么使用不同投影?
- 18. Q018 为什么使用 Dot-Product Attention, 而不是任意 MLP 相似度?
- 19. Q019 Self-Attention 的时间复杂度到底是多少?
- 20. Q020 Causal Mask 的矩阵结构是什么?
- 21. Q021 Padding Mask 与 Causal Mask 如何组合?
- 22. Q022 Attention Dropout 通常加在哪里?
- 23. Q023 Attention Weight 能否直接作为模型解释?
- 24. Q024 为什么 Linear Attention 不能简单替代 Softmax Attention?
- 25. Q025 写出经典 Sinusoidal Position Encoding

第三章 位置编码与 RoPE 从“Attention 本身没有顺序”推导 Absolute / Relative / RoPE / ALiBi 与长上下文外推。

- 26. Q026 为什么原始 Transformer 使用固定 Sin/Cos?
- 27. Q027 Learned Position Embedding 有什么限制?
- 28. Q028 Absolute Position 与 Relative Position 的区别是什么?
- 29. Q029 RoPE 的核心数学是什么?
- 30. Q030 为什么 RoPE 通常作用在 Q/K 而不是 V?
- 31. Q031 RoPE 为什么会遇到长上下文外推问题?
- 32. Q032 ALiBi 与 RoPE 的思想有什么区别?
- 33. Q033 Position Interpolation 的基本思想是什么?
- 34. Q034 技术上支持 64K 与真正有效 64K 有什么区别?
- 35. Q035 Pre-LN 与 Post-LN 有什么区别?

第四章 Norm、Residual 与 FFN 理解深层 Transformer 为什么能训练, 以及 RMSNorm、SwiGLU 等现代模块为什么出现。

- 36. Q036 为什么现代 LLM 普遍偏向 Pre-Norm?
- 37. Q037 LayerNorm 到底沿哪个维度计算?
- 38. Q038 RMSNorm 与 LayerNorm 有什么区别?
- 39. Q039 FFN 为什么不可缺少?
- 40. Q040 为什么经典 Transformer 常用 $d_{ff} \approx 4d_{model}$?
- 41. Q041 GELU 为什么比 ReLU 常见?
- 42. Q042 SwiGLU 是什么, 为什么现代 LLM 常用?
- 43. Q043 为什么深层 Transformer 对初始化特别敏感?
- 44. Q044 为什么 Input Embedding 与 LM Head 经常 Weight Tying?
- 45. Q045 Encoder-only、Decoder-only、Encoder-Decoder 的本质区别?

第五章 BERT、GPT 与 Encoder-Decoder 范式 把预训练目标、Mask 结构与任务形态联系起来, 而不是背模型名字。

- 46. Q046 BERT 为什么可以双向?
- 47. Q047 BERT 为什么用 MLM, 而 GPT 用 CLM?

- 48. Q048 BERT 的 NSP 是什么，为什么后来常被移除？
- 49. Q049 GPT 的 Causal LM 训练目标是什么？
- 50. Q050 为什么自回归模型训练时还能并行？
- 51. Q051 为什么 Decoder-only 成为通用 LLM 主流之一？
- 52. Q052 Cross-Attention 为什么适合机器翻译？
- 53. Q053 Teacher Forcing 是什么？
- 54. Q054 Exposure Bias 是什么？
- 55. Q055 Decoder LM 的 label 为什么必须 shift？

第六章 训练、优化与 Debug 覆盖 label shift、NaN、AdamW、warmup、混合精度、梯度与分布式训练。

- 56. Q056 Padding token 为什么不能参与 loss？
- 57. Q057 Transformer loss 变成 NaN，如何系统排查？
- 58. Q058 为什么训练早期常用 Learning Rate Warmup？
- 59. Q059 Adam 与 AdamW 的核心区别？
- 60. Q060 Gradient Clipping 为什么有用？
- 61. Q061 Gradient Accumulation 是否完全等价于大 Batch？
- 62. Q062 FP16 与 BF16 为什么训练表现不同？
- 63. Q063 训练正常但验证每次结果不同，先查什么？
- 64. Q064 Transformer 初始化不合理有哪些症状？
- 65. Q065 固定训练 FLOPs，模型更大还是 token 更多？
- 66. Q066 Transformer 多 GPU 有哪些并行方式？
- 67. Q067 Decoder-only LLM 的生成过程是什么？

第七章 自回归推理与 KV Cache 从增量解码推导 KV Cache、GQA/MQA、采样、TTFT/TPOT 与 serving 内存。

- 68. Q068 KV Cache 缓存了什么，为什么不缓存 Q？
- 69. Q069 KV Cache 内存如何估算？
- 70. Q070 为什么训练不需要像 decode 一样使用 KV Cache？
- 71. Q071 Prefill 与 Decode 的瓶颈有什么不同？
- 72. Q072 MHA、MQA、GQA 的区别？
- 73. Q073 Greedy、Beam Search 与 Sampling 如何选择？
- 74. Q074 Temperature、Top-k、Top-p 分别做什么？
- 75. Q075 为什么模型会陷入重复生成？
- 76. Q076 LLM Serving 的关键 latency 指标有哪些？
- 77. Q077 Continuous Batching 为什么适合 LLM？
- 78. Q078 PagedAttention 解决什么问题？
- 79. Q079 Speculative Decoding 为什么能加速且保持目标分布？
- 80. Q080 FlashAttention 到底优化了什么？

第八章 长上下文、高性能 Attention 与 MoE 理解 FlashAttention 的 IO 本质、Sparse/Linear Attention、MoE、量化与长上下文瓶颈。

- 81. Q081 FlashAttention 为什么不需要存完整 Attention Matrix？
- 82. Q082 Online Softmax 如何分块仍保持精确？
- 83. Q083 Sparse Attention 如何降低复杂度？
- 84. Q084 Linear Attention 与 FlashAttention 最大区别？
- 85. Q085 长上下文 Transformer 至少有哪些瓶颈？
- 86. Q086 Sliding Window Attention 的优势与损失？
- 87. Q087 MoE 为什么能在相近每-token FLOPs 下增加参数量？
- 88. Q088 MoE 最大训练难点是什么？

89. Q089 Quantization 为什么对 Transformer 推理重要?

90. Q090 Coding: 从零实现 Multi-Head Attention

第九章 Coding / Debug 真题型 将公式转成正确代码, 并通过不变量、对照实验和单元测试快速定位实现错误。

91. Q091 Coding: 下面的 Attention 有哪些 bug?

92. Q092 Coding: 给 Transformer 加 KV Cache

93. Q093 给你几百行 Transformer 代码, 如何系统 Debug?

94. Q094 Coding: 如何验证 causal mask 没有未来泄漏?

95. Q095 Coding: 如何验证 KV Cache 实现正确?

96. Q096 Coding: 为什么 transpose 后 view 经常报错或 silently 出问题?

97. Q097 Coding: 如何处理全 Mask 行导致的 NaN?

98. Q098 Coding: 如何做 Tiny Overfit Test?

99. Q099 Coding: 如何为 Attention 写 Shape Assertions?

100. Q100 System Design: 用 Transformer 设计文本分类系统

第一章 Transformer 总体架构与设计动机

从 RNN 的瓶颈出发，建立 Encoder、Decoder、Residual Stream、Shape 与信息流的整体图景。

题号	题目	难度
Q001	为什么 Transformer 能取代 RNN/LSTM?	2/5
Q002	请完整描述一个 Transformer Encoder Layer	2/5
Q003	请完整描述 Transformer Decoder	2/5
Q004	Residual Connection 到底解决什么问题?	3/5
Q005	为什么 Transformer 更常用 LayerNorm 而不是 BatchNorm?	2/5
Q006	手写 Transformer 时最重要的 Tensor Shape 链是什么?	2/5
Q007	标准 Multi-Head Attention 参数量如何估算?	2/5
Q008	Self-Attention 与 Cross-Attention 的本质区别是什么?	2/5
Q009	Attention 与 FFN 分别承担什么功能?	2/5
Q010	如果完全删除 Position Encoding, 会发生什么?	3/5

Q001 为什么 Transformer 能取代 RNN/LSTM?

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 判断你是否从计算图、并行性和长距离依赖路径理解 Transformer 的设计动机，而不是只会说“Attention 更强”。

30 秒答案

RNN 的隐藏状态具有严格时间依赖，训练沿序列维度串行；Self-Attention 可以一次计算所有 token 间的相关性，并把任意两个 token 的信息传播路径缩短到一层。代价是标准 Attention 在长序列上引入 $O(T^2)$ 的 pairwise 计算与中间存储。

$$h_t = f(h_{t-1}, x_t) \text{ vs. } \text{Attention}(X) = \text{softmax}(QK^T / \sqrt{d_k})V$$

深度拆解

- RNN 的关键递推是 $h_t = f(h_{t-1}, x_t)$ ，因此 h_t 不能在 h_{t-1} 未完成时计算。
- Self-Attention 直接形成 $T \times T$ 的交互图，训练阶段序列维度可并行。
- 长距离依赖的“最短路径长度”由 RNN 的 $O(T)$ 降到单层 Attention 的 $O(1)$ 。
- Transformer 不是“无序列依赖”：自回归解码仍然逐 token 串行；它主要解除训练阶段的递推依赖。
- 真正的 trade-off 是：更好的并行性与全局交互，换来 Attention 在长上下文下的二次复杂度与显存/带宽压力。

常见失分点 把“训练并行”误说成“生成也完全并行”；或者只回答“能看到全局”。

继续追问 如果序列长度 T 很短、hidden size d 很大，Attention 一定是主要算力瓶颈吗？请写出 $O(Td^2+T^2d)$ 并讨论主导项。

一语中的：RNN 的隐藏状态具有严格时间依赖，训练沿序列维度串行；Self-Attention 可以一次计算所有 token 间的相关性，并把任意两个 token 的信息传播路径缩短到一层。

Q002 请完整描述一个 Transformer Encoder Layer

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察你能否把信息流、模块顺序与现代 Pre-Norm 写清楚。

30 秒答案

现代常见 Pre-Norm Encoder block 可以写成 $H=X+\text{Attention}(\text{Norm}(X))$, $Y=H+\text{FFN}(\text{Norm}(H))$ 。Attention 负责 token 之间的信息交换，FFN 负责每个 token 内部的非线性特征变换。

$$H=X+\text{Attn}(\text{Norm}(X)); Y=H+\text{FFN}(\text{Norm}(H))$$

深度拆解

- 输入通常是 $[B, T, d_{\text{model}}]$ 。
- Self-Attention 的 Q/K/V 均来自同一 X ，因此是 self-attention。
- Residual path 保留原表示并提供直接梯度路径。
- Norm 放在子层前称 Pre-Norm；原始 2017 Transformer 更接近 Post-Norm。
- FFN 对每个 token 独立应用，因此不会直接产生 token-to-token mixing。

常见失分点 只罗列“Attention + FFN + LayerNorm”，却说不清顺序，也不知道 Pre-LN/Post-LN 的区别。

继续追问 如果把 FFN 删掉，只堆 Attention，会损失什么表达能力？

一语中的：现代常见 Pre-Norm Encoder block 可以写成 $H=X+\text{Attention}(\text{Norm}(X))$, $Y=H+\text{FFN}(\text{Norm}(H))$ 。

Q003 请完整描述 Transformer Decoder

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 区分 masked self-attention、cross-attention 与 FFN，并理解 Decoder-only 与经典 Encoder-Decoder 的差异。

30 秒答案

经典 Transformer Decoder 先做 causal self-attention，再用 decoder hidden state 作为 Query 对 encoder output 做 cross-attention，最后经过 FFN。Decoder-only LLM 通常没有 cross-attention，只保留 causal self-attention 与 MLP。

深度拆解

- Masked self-attention 保证位置 t 只能使用 $\leq t$ 的 token。

- Cross-attention 中 Q 来自 Decoder，K/V 来自 Encoder，是“输出序列向输入序列检索”。
- 训练时 teacher forcing 允许整段并行；推理时仍需增量生成。
- Decoder-only 把 prompt、上下文、答案统一成一个 token stream。

常见失分点 把“Decoder 必有 Cross-Attention”当成现代 LLM 的通用结论。

继续追问 为什么 Encoder-Decoder 翻译模型通常能预先缓存 Encoder 的 K/V？

一语中的：经典 Transformer Decoder 先做 causal self-attention，再用 decoder hidden state 作为 Query 对 encoder output 做 cross-attention，最后经过 FFN。

Q004 Residual Connection 到底解决什么问题？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 是否理解深层网络中的优化路径，而不是把 residual 当成“防止信息丢失”的口号。

30 秒答案

Residual 令 $x_{l+1}=x_l+F(x_l)$ 。对 x_l 求导时包含恒等项 I，因此即使 F 分支局部梯度很小，仍有直接梯度路径；同时每层可以学习对 residual stream 的增量更新。

$$x_{l+1}=x_l+F(x_l), \quad \partial x_{l+1}/\partial x_l=I+\partial F/\partial x_l$$

深度拆解

- 梯度： $\partial x_{l+1}/\partial x_l=I+\partial F/\partial x_l$ 。
- Residual 不保证永不梯度爆炸/消失；尺度、初始化、Norm 位置仍重要。
- 现代 LLM 可以把 hidden state 理解为贯穿所有层的 residual stream，多种子层不断向其中“写入”信息。

常见失分点 说成“Residual 让梯度恒等于 1”。实际上是 $I + \text{Jacobian}$ 。

继续追问 为什么深度增加后 residual branch 的初始化尺度通常要更谨慎？

一语中的：Residual 令 $x_{l+1}=x_l+F(x_l)$ 。

Q005 为什么 Transformer 更常用 LayerNorm 而不是 BatchNorm？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察归一化统计量、变长序列与 train/inference 一致性。

30 秒答案

LayerNorm 对每个 token 的 hidden dimension 独立归一化，不依赖 batch 统计量；因此对小 batch、变长序列和分布式训练更稳定，训练与推理行为也一致。

$$\text{LN}(x)=\gamma\cdot(x-\mu)/\sqrt{(\sigma^2+\epsilon)}+\beta$$

深度拆解

- 输入 $[B,T,d]$ 时，LayerNorm 通常沿最后的 d 维计算均值与方差。
- BatchNorm 的统计量与 batch composition 有关，小 batch 和跨设备同步都更麻烦。

- Transformer 的 token 数与 padding 情况变化大，LayerNorm 的样本内归一化更自然。

常见失分点 误以为 LayerNorm 是沿 T 维归一化。

继续追问 如果 batch size=1，BatchNorm 和 LayerNorm 的差异会如何表现？

一语中的：LayerNorm 对每个 token 的 hidden dimension 独立归一化，不依赖 batch 统计量；因此对小 batch、变长序列和分布式训练更稳定，训练与推理行为也一致。

Q006 手写 Transformer 时最重要的 Tensor Shape 链是什么？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 代码面试的基础不变量：你是否能在不反复试错的情况下写对 reshape/transpose。

30 秒答案

从 $X:[B,T,d]$ 出发，投影后 $Q/K/V:[B,T,d]$ ，拆头得到 $[B,H,T,D_h]$ ； $score=QK^T:[B,H,T,T]$ ；乘 V 后回到 $[B,H,T,D_h]$ ，再合并成 $[B,T,d]$ 。

$$[B,T,d] \rightarrow [B,H,T,D_h] \rightarrow [B,H,T,T] \rightarrow [B,H,T,D_h] \rightarrow [B,T,d]$$

深度拆解

- 必须满足 $d=H \cdot D_h$ 。
- $K.transpose(-2,-1)$ 交换的是 T 与 D_h 。
- Softmax 通常沿最后一个 key 维度。
- transpose 后若继续 view，通常需要 contiguous() 或 reshape。

常见失分点 把 $[B,T,H,D_h]$ 直接与另一个未转置张量相乘；或 $softmax(dim=1)$ 。

继续追问 如果 K/V head 数少于 Q head（GQA），shape 如何变化和 broadcast/group？

一语中的：从 $X:[B,T,d]$ 出发，投影后 $Q/K/V:[B,T,d]$ ，拆头得到 $[B,H,T,D_h]$ ； $score=QK^T:[B,H,T,T]$ ；乘 V 后回到 $[B,H,T,D_h]$ ，再合并成 $[B,T,d]$ 。

Q007 标准 Multi-Head Attention 参数量如何估算？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 判断你是否能区分“head 数量”和“总投影维度”的关系。

30 秒答案

若 $d_{model}=d$ 且 $Q/K/V/O$ 都是 $d \times d$ 线性层，忽略 bias 参数约为 $4d^2$ 。只要总 hidden size d 固定，增加 head 数并不会线性增加这四个投影的总参数量。

$$Params(MHA) \approx 4d^2$$

深度拆解

- W_Q, W_K, W_V, W_O 各约 d^2 。
- head 数改变的是 d 如何切分成 H 个 D_h ，而不是一定改变总投影维度。

- GQA/MQA 会减少 K/V projection 输出维度和 KV Cache，但具体参数量取决于实现。

常见失分点 回答“8 个头就是 8 倍参数”。

继续追问 若 $H_q=32$ 、 $H_{kv}=8$ ，Q/K/V 的权重矩阵 shape 如何写？

一语中的：若 $d_{\text{model}}=d$ 且 Q/K/V/O 都是 $d \times d$ 线性层，忽略 bias 参数约为 $4d^2$ 。

Q008 Self-Attention 与 Cross-Attention 的本质区别是什么？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 是否能用 Q/K/V 的来源解释，而不是用模型名字解释。

30 秒答案

Self-Attention 的 Q/K/V 来自同一序列；Cross-Attention 的 Q 来自目标/查询序列，而 K/V 来自另一个源序列。后者可以理解为可微分的内容寻址或 soft retrieval。

深度拆解

- Self-attention 建模序列内部依赖。
- Cross-attention 建模两个序列/模态之间的对齐。
- 在翻译中，decoder 每个位置用自身状态作为 Query，对 encoder memory 进行动态读取。

常见失分点 把 Cross-Attention 说成“Q、K、V 都来自 Encoder”。

继续追问 多模态模型中，图像 token 与文本 token 可以在哪些位置使用 cross-attention？

一语中的：Self-Attention 的 Q/K/V 来自同一序列；Cross-Attention 的 Q 来自目标/查询序列，而 K/V 来自另一个源序列。

Q009 Attention 与 FFN 分别承担什么功能？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解 Transformer 为什么既需要 token mixing 又需要 channel/feature mixing。

30 秒答案

Attention 主要决定“从哪些 token 读取信息”，FFN 主要决定“对每个 token 的特征做什么非线性计算”。前者跨 token，后者逐 token。

Attention: token mixing | FFN: feature mixing

深度拆解

- Attention 输出是对 V 的输入依赖加权组合。
- FFN 通常是 $d \rightarrow d_{\text{ff}} \rightarrow d$ 的两层或 gated MLP。
- 即使 Attention 完美聚合上下文，没有强非线性 MLP 也会限制特征变换能力。

常见失分点 说“FFN 用来增加参数”而没有解释功能。

继续追问 为什么现代 LLM 中 FFN 参数量常常比 Attention 更多？

一语中的： Attention 主要决定“从哪些 token 读取信息”，FFN 主要决定“对每个 token 的特征做什么非线性计算”。

Q010 如果完全删除 Position Encoding，会发生什么？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 从 permutation equivariance 推导位置编码的必要性。

30 秒答案

没有任何位置机制时，Self-Attention 对输入 token 的排列是置换等变的：交换输入 token，输出会按同样方式交换，但模型本身无法知道“第几个”“前后关系”和距离。位置机制把集合结构重新变成序列结构。

深度拆解

- Q/K/V 只由 token representation 线性变换得到，不包含顺序。
- 位置可以通过 embedding 加法、relative bias、RoPE 等不同方式注入。
- Causal mask 本身提供“是否在过去”的粗顺序约束，但不等价于完整位置表示。

常见失分点 误以为 causal mask 已经完全替代位置编码。

继续追问 只有 causal mask、没有显式 positional encoding 的 decoder 能学到哪些位置信息，哪些会受限？

一语中的： 没有任何位置机制时，Self-Attention 对输入 token 的排列是置换等变的：交换输入 token，输出会按同样方式交换，但模型本身无法知道“第几个”“前后关系”和距离。

第二章 Attention 数学与实现细节

从点积方差、Softmax 数值稳定性、Mask 到复杂度，要求能手推也能落地实现。

题号	题目	难度
Q011	写出 Scaled Dot-Product Attention，并逐项解释	2/5
Q012	为什么必须除以 $\sqrt{d_k}$?	3/5
Q013	为什么 Mask 要加在 Softmax 之前?	2/5
Q014	Softmax 如何避免数值溢出?	2/5
Q015	为什么需要 Multi-Head Attention?	3/5
Q016	为什么通常设置 $d_{\text{head}}=d_{\text{model}}/H$?	2/5
Q017	Q、K、V 为什么使用不同投影?	3/5
Q018	为什么使用 Dot-Product Attention，而不是任意 MLP 相似度?	3/5
Q019	Self-Attention 的时间复杂度到底是多少?	3/5
Q020	Causal Mask 的矩阵结构是什么?	2/5
Q021	Padding Mask 与 Causal Mask 如何组合?	3/5
Q022	Attention Dropout 通常加在哪里?	2/5
Q023	Attention Weight 能否直接作为模型解释?	3/5
Q024	为什么 Linear Attention 不能简单替代 Softmax Attention?	4/5
Q025	写出经典 Sinusoidal Position Encoding	2/5

Q011 写出 Scaled Dot-Product Attention，并逐项解释

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 这是 Transformer 面试的“九九乘法表”。需要公式、shape 和直觉同时正确。

30 秒答案

$\text{Attention}(Q,K,V)=\text{softmax}(QK^T/\sqrt{d_k}+M)V$ 。 QK^T 产生 query-key 匹配分数；缩放控制 logit 方差；M 注入 causal/padding 等约束；Softmax 形成归一化权重；最后对 V 做加权读取。

$$\text{Attention}(Q,K,V)=\text{softmax}(QK^T/\sqrt{d_k}+M)V$$

深度拆解

- $Q:[...,T_q,d_k]$, $K:[...,T_k,d_k]$, $V:[...,T_k,d_v]$ 。
- score shape 是 $[...,T_q,T_k]$ 。
- mask 必须和 score 可广播，并确保非法位置 softmax 后概率为 0。

- 最终输出 shape 为 [...,T_q,d_v]。

代码 / 伪代码骨架

```
def attention(q, k, v, mask=None):
    scores = q @ k.transpose(-2, -1) / math.sqrt(q.size(-1))
    if mask is not None:
        scores = scores.masked_fill(~mask, float("-inf"))
    p = torch.softmax(scores, dim=-1)
    return p @ v
```

常见失分点 公式里漏掉缩放、mask 或把 V 写进 Softmax。

继续追问 当 T_q=1 时，这个公式对应自回归 decode 的哪一步？

一语中的： Attention(Q,K,V)=softmax(QK^T/√d_k + M)V。

Q012 为什么必须除以 √d_k?

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 要求真正推导 dot-product 方差与 Softmax 饱和，而不是背一句“稳定梯度”。

30 秒答案

若 q_i、k_i 独立、均值 0、方差 1，则 $q \cdot k = \sum q_{ik} i$ 的方差约为 d_k，标准差约为 √d_k。维度增大时 logit 绝对值会变大，Softmax 更尖锐、梯度更容易进入饱和区。除以 √d_k 把典型尺度拉回 O(1)。

$$\text{Var}(q \cdot k) \approx d_k \Rightarrow \text{std} \approx \sqrt{d_k}$$

深度拆解

- 关键是点积由 d_k 个随机项求和。
- 如果 q/k 的初始化方差不是 1，实际 logit 方差也会相应变化，但 √d_k 是结构上的主缩放。
- 缩放不是为了“把概率除小”，而是发生在 Softmax 前的 logits。

常见失分点 只说“防止梯度爆炸”，没有说明 Softmax 饱和。

继续追问 如果改成除以 d_k，会发生什么？温度会如何变化？

一语中的： 若 q_i、k_i 独立、均值 0、方差 1，则 $q \cdot k = \sum q_{ik} i$ 的方差约为 d_k，标准差约为 √d_k。

Q013 为什么 Mask 要加在 Softmax 之前？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察概率归一化的数学正确性与常见实现 bug。

30 秒答案

要让非法位置概率严格为 0，应在 logits 上把它们设为 $-\infty$ （或足够小的数），再做 Softmax。若先 Softmax 再把概率乘 0，剩余合法概率之和小于 1，已经不是原定义的归一化注意力。

$$\text{scores} \leftarrow \text{scores} + \mathbf{M}, \mathbf{M}_{ij} \in \{0, -\infty\}$$

深度拆解

- $\text{softmax}(-\infty)=0$ 。
- padding mask 与 causal mask 可以在 logits 级组合。
- FP16 中使用极小有限值时要注意数值范围，现代框架通常封装了安全 kernel。

常见失分点 先 softmax 再乘 mask。

继续追问 如果一整行所有位置都被 mask，会发生什么？如何避免 NaN？

一语中的： 要让非法位置概率严格为 0，应在 logits 上把它们设为 $-\infty$ （或足够小的数），再做 Softmax。

Q014 Softmax 如何避免数值溢出？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解 log-sum-exp trick，这是 Attention 稳定性与 FlashAttention online softmax 的基础。

30 秒答案

利用 softmax 对整体平移不变：先减去一行最大值 m ，再计算 $\exp(x_i - m)$ ，最大指数变成 $\exp(0)=1$ ，显著降低 overflow 风险。

$$\text{softmax}(x_i) = \exp(x_i - m) / \sum_j \exp(x_j - m), m = \max_j x_j$$

深度拆解

- $\text{softmax}(x) = \text{softmax}(x - c)$ 。
- 减最大值同时保留完全相同的概率结果。
- 这也是在线/分块 Softmax 可以通过维护 running max 和 normalization factor 精确合并的基础。

常见失分点 只说“框架会自动处理”，却说不出公式。

继续追问 为什么减去均值不如减最大值直接？

一语中的： 利用 softmax 对整体平移不变：先减去一行最大值 m ，再计算 $\exp(x_i - m)$ ，最大指数变成 $\exp(0)=1$ ，显著降低 overflow 风险。

Q015 为什么需要 Multi-Head Attention？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察表示子空间、多关系建模与工程 trade-off。

30 秒答案

Multi-Head 让模型在多个不同的 Q/K/V 投影子空间中并行计算注意力，再合并。它增加的是“不同匹配子空间”的表达能力，而不是简单把一个 Attention 重复多次。

$$\text{MHA} = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W_O$$

深度拆解

- 第 h 个 head 有独立 $W_Q^h/W_K^h/W_V^h$ （或等价的分块大矩阵）。
- 不同 head 可能学习不同关系，但不应把每个 head 强行解释成一个固定语法角色。
- 固定 d_{model} 时，更多 head 意味着更小 d_{head} ，存在容量与硬件效率 trade-off。

常见失分点 声称“每个 head 一定负责一个语法关系”。

继续追问 如果把所有 head 的 $W_Q/W_K/W_V$ 强制共享，Multi-Head 还剩下什么意义？

一语中的： Multi-Head 让模型在多个不同的 Q/K/V 投影子空间中并行计算注意力，再合并。

Q016 为什么通常设置 $d_{\text{head}} = d_{\text{model}}/H$?

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察对总维度、计算量和参数预算的控制。

30 秒答案

若每个 head 都维持完整 d_{model} ， H 个 head 的中间维度会膨胀 H 倍。将每头维度设为 d/H ，使 concat 后仍回到 d ，从而在增加多个子空间的同时控制总宽度与成本。

深度拆解

- 满足 $H \cdot d_{\text{head}} = d_{\text{model}}$ 。
- 硬件实现还会偏好某些 d_{head} （如 64/128）以匹配 tensor core/kernel。
- GQA 时 Q head 与 KV head 数可以不同，但每个 head dimension 通常一致。

常见失分点 把 d_{head} 理解为一个完全独立超参数而不考虑总 hidden size。

继续追问 如果 d_{model} 不能被 H 整除，工程上会怎么处理？

一语中的： 若每个 head 都维持完整 d_{model} ， H 个 head 的中间维度会膨胀 H 倍。

Q017 Q、K、V 为什么使用不同投影？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解“匹配空间”和“内容空间”的角色分离。

30 秒答案

Q 表示“我要找什么”，K 表示“我如何被匹配”，V 表示“真正被读取的内容”。独立投影允许相似度计算空间与信息承载空间不同，从而提升表达灵活性。

深度拆解

- QK^T 只用于决定权重；V 不直接参与匹配。
- 如果 $Q=K=V=X$ 且没有投影，模型只能在原表示空间做固定形式的相似度与读取。
- Cross-Attention 更清楚地体现 Q 与 K/V 的角色分工。

常见失分点 解释成“三个矩阵只是为了增加参数”。

继续追问 能否让 $K=V$ 共享投影？会带来什么容量与效率变化？

一语中的： Q 表示“我要找什么”， K 表示“我如何被匹配”， V 表示“真正被读取的内容”。

Q018 为什么使用 Dot-Product Attention，而不是任意 MLP 相似度？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察数学表达与 GPU GEMM 友好性。

30 秒答案

Dot-product 可以批量写成 QK^T ，本质是大规模 GEMM，GPU 对此高度优化；同时它提供简单的双线性相似度。Additive attention 也可表达复杂匹配，但 pairwise MLP 往往更难高效批量化。

深度拆解

- 工程上的高吞吐是 Transformer 成功的重要因素之一。
- 大矩阵乘法可充分利用 tensor cores。
- “更复杂的 score function” 不等于 wall-clock 更优。

常见失分点 只从“余弦相似度”解释，忽略 Q/K 未归一化且 GPU 计算优势。

继续追问 如果先对 Q/K 做 L2 normalization，注意力变成什么性质？

一语中的： Dot-product 可以批量写成 QK^T ，本质是大规模 GEMM，GPU 对此高度优化；同时它提供简单的双线性相似度。

Q019 Self-Attention 的时间复杂度到底是多少？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 避免机械回答 $O(T^2)$ ，要求把投影和 pairwise 两部分写全。

30 秒答案

一个标准 Attention block 的主要成本可粗分为线性投影 $O(Td^2)$ 与 score/value 交互 $O(T^2d)$ 。因此总量级约 $O(Td^2+T^2d)$ 。 T 较小、 d 很大时投影/MLP 可能更主导； T 很长时二次项迅速占优。

$$\text{Cost} \approx O(Td^2 + T^2d)$$

深度拆解

- $Q/K/V/O$ 的 GEMM 合计与 Td^2 同阶。
- QK^T 与 AV 都与 T^2d 同阶。
- 空间上显式保存 attention matrix 需要 $O(T^2)$ ，但 FlashAttention 可避免 materialize 完整矩阵。

常见失分点 把 FlashAttention 误说成把数学 FLOPs 从 $O(T^2)$ 降到 $O(T)$ 。

继续追问 给定 $T=4K$ 、 $d=4K$ ，粗略比较 Td^2 与 T^2d 的量级。

一语中的： 一个标准 Attention block 的主要成本可粗分为线性投影 $O(Td^2)$ 与 score/value 交互 $O(T^2d)$ 。

Q020 Causal Mask 的矩阵结构是什么？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 判断是否真正理解“位置 t 不能读取未来位置”。

30 秒答案

对 $\text{score}(i,j)$ ，当 $j>i$ 时设为 $-\infty$ ，其余为 0。矩阵表现为上三角被屏蔽、下三角（含对角线）可见。这样训练时虽然整段并行，位置 i 的计算图仍只依赖过去。

$$M_{ij}=0 \ (j \leq i), -\infty \ (j > i)$$

深度拆解

- 对角线是否允许取决于是否在预测前做了 shift，但标准 causal self-attention 通常允许当前位置读取当前输入 token。
- left padding 会让 mask 组合更复杂。
- mask 方向写反是常见 silent bug：loss 可能下降但任务异常。

常见失分点 把上三角/下三角方向记反。

继续追问 如何设计一个行为级单元测试证明没有未来信息泄漏？

一语中的： 对 $\text{score}(i,j)$ ，当 $j>i$ 时设为 $-\infty$ ，其余为 0。

Q021 Padding Mask 与 Causal Mask 如何组合？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察真实 batch 中 padding、广播和掩码语义。

30 秒答案

Padding mask 禁止读取无效 pad token；causal mask 禁止读取未来 token。最终可在 logits 上做逻辑 AND（布尔 mask）或把两类 additive mask 相加。

深度拆解

- 右 padding 与 left padding 的有效区域不同。
- 在 decoder-only batch 中，position ids、causal mask、padding mask 往往需要一起验证。
- FlashAttention/SDPA 等 fused kernel 可能使用不同 mask API，语义必须对齐。

常见失分点 把 padding mask 仅用于 loss，却忘记 Attention 仍会读取 pad hidden state。

继续追问 为什么 left padding 对批量生成常见，但会让 position id 更容易出错？

一语中的：Padding mask 禁止读取无效 pad token；causal mask 禁止读取未来 token。

Q022 Attention Dropout 通常加在哪里？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察训练/推理模式与概率结构。

30 秒答案

常见做法是在 Softmax 后的 attention probability 上做 dropout，再与 V 相乘；此外 output projection 或 residual branch 也可能有 dropout。推理时 model.eval() 必须关闭随机 dropout。

深度拆解

- attention dropout 随机丢弃一部分连接权重并重新缩放。
- 不同模型 recipe 可能将 dropout 设置为 0，尤其大模型预训练。
- 不要把 dropout 与 mask 混为一谈：mask 是结构约束，dropout 是随机正则化。

常见失分点 验证时忘记 model.eval()，导致每次结果不同。

继续追问 为什么超大规模预训练中常见 dropout 逐渐减小甚至为 0？

一语中的：常见做法是在 Softmax 后的 attention probability 上做 dropout，再与 V 相乘；此外 output projection 或 residual branch 也可能有 dropout。

Q023 Attention Weight 能否直接作为模型解释？

难度 3/5 | 中频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察你是否知道相关性、信息流与因果归因的差别。

30 秒答案

Attention weight 是内部路由权重，能提供一些观察，但不能直接等同于“模型做出预测的因果原因”。Residual stream、后续层、FFN 与多个 head 都会继续改变信息。

深度拆解

- 某个 token attention 高，不意味着删除该 token 一定对最终输出影响最大。
- 可结合 ablation、gradient attribution、causal intervention 等更强方法。
- 解释性结论需要区分“可视化”与“因果证据”。

常见失分点 说“注意力热力图就是解释”。

继续追问 如果两个 attention pattern 不同但最终 hidden state 相似，说明什么？

一语中的：Attention weight 是内部路由权重，能提供一些观察，但不能直接等同于“模型做出预测的因果原因”。

Q024 为什么 Linear Attention 不能简单替代 Softmax Attention？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解复杂度降低与模型表达/硬件效率之间的 trade-off。

30 秒答案

Linear Attention 常通过 kernel feature map 重排计算，使 K^TV 先聚合，从而避免显式 $T \times T$ ；但它改变了标准 Softmax Attention 的函数族，可能损失表达，并且低 FLOPs 不一定等于高 GPU 利用率。

深度拆解

- 标准 Softmax Attention 能对每个 query 动态形成完整 token 分布。
- 线性化需要特定核分解或近似。
- 现代硬件上，FlashAttention 把 exact attention 做得很快，使一些近似方法的 wall-clock 优势缩小。

常见失分点 认为任何 $O(T)$ 理论算法必然比 FlashAttention 更快。

继续追问 在什么场景下线性/状态空间模型更可能显示优势？

一语中的： Linear Attention 常通过 kernel feature map 重排计算，使 K^TV 先聚合，从而避免显式 $T \times T$ ；但它改变了标准 Softmax Attention 的函数族，可能损失表达，并且低 FLOPs 不一定等于高 GPU 利用率。

Q025 写出经典 Sinusoidal Position Encoding

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 要求知道原始 Transformer 的位置注入方式与多频率设计。

30 秒答案

原始 Transformer 使用固定正弦/余弦函数，在不同维度放置不同频率，使每个位置具有确定性向量并可计算任意位置。

$$PE(pos, 2i) = \sin(pos/10000^{(2i/d)}); PE(pos, 2i+1) = \cos(...)$$

深度拆解

- 偶数维使用 sin，奇数维使用 cos。
- 频率按 10000 的指数尺度变化。
- 位置向量与 token embedding 相加。
- 固定位置编码没有额外可训练参数。

常见失分点 只会说“sin/cos 让模型知道顺序”，写不出频率结构。

继续追问 为什么一组多频正弦比只用 pos/d 的线性标量更有表达力？

一语中的： 原始 Transformer 使用固定正弦/余弦函数，在不同维度放置不同频率，使每个位置具有确定性向量并可计算任意位置。

第三章 位置编码与 RoPE

从“Attention 本身没有顺序”推导 Absolute / Relative / RoPE / ALiBi 与长上下文外推。

题号	题目	难度
Q026	为什么原始 Transformer 使用固定 Sin/Cos?	2/5
Q027	Learned Position Embedding 有什么限制?	2/5
Q028	Absolute Position 与 Relative Position 的区别是什么?	3/5
Q029	RoPE 的核心数学是什么?	4/5
Q030	为什么 RoPE 通常作用在 Q/K 而不是 V?	3/5
Q031	RoPE 为什么会遇到长上下文外推问题?	4/5
Q032	ALiBi 与 RoPE 的思想有什么区别?	3/5
Q033	Position Interpolation 的基本思想是什么?	3/5
Q034	技术上支持 64K 与真正有效 64K 有什么区别?	4/5
Q035	Pre-LN 与 Post-LN 有什么区别?	3/5

Q026 为什么原始 Transformer 使用固定 Sin/Cos?

难度 2/5 | 高频 | 面试形式: 概念 / 推导 / 追问

面试官在考什么 考察设计权衡，而非把固定编码误说成唯一选择。

30 秒答案

固定 Sin/Cos 无需学习位置表，能为任意位置生成编码，并具有规则的相对位移结构。它是一个工程/归纳偏置选择，而不是“位置编码不能训练”。

深度拆解

- learned absolute embedding 也是常见方案。
- 固定函数理论上更方便外推到未见位置，但模型是否真正能利用仍受训练分布限制。
- 现代 LLM 更常见 RoPE 等相对位置机制。

常见失分点 说“固定 PE 一定比 learned PE 泛化更好”。

继续追问 为什么有些 learned absolute PE 模型也能在训练长度内表现很好?

一语中的: 固定 Sin/Cos 无需学习位置表，能为任意位置生成编码，并具有规则的相对位移结构。

Q027 Learned Position Embedding 有什么限制？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解参数表、最大长度与外推问题。

30 秒答案

Learned PE 通常维护 $P \in \mathbb{R}^{T_{\max} \times d}$ 。训练长度之外没有直接学到对应位置向量，因此 extension/extrapolation 较困难；但在固定长度内具有更高自由度。

深度拆解

- 参数量约 $T_{\max} \cdot d$ 。
- 扩展位置表可插值或继续训练，但不是无成本。
- 绝对 PE 与相对位置关系需要网络间接学习。

常见失分点 把“无法超过 $T_{\max}$ ”说成代码绝对不能运行；实际上可以改表，但泛化另说。

继续追问 如果把 4K 的 learned table 插值到 8K，可能出现哪些问题？

一语中的： Learned PE 通常维护 $P \in \mathbb{R}^{T_{\max} \times d}$ 。

Q028 Absolute Position 与 Relative Position 的区别是什么？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 从关系建模角度说明为什么相对位置很自然。

30 秒答案

Absolute position 表示 token 自身的坐标 p_i ；relative position 直接建模 $i-j$ 或距离/方向。语言中的依赖往往与相对距离更相关，因此 relative bias、RoPE 等方式能更直接地影响 token pair 的注意力。

深度拆解

- 绝对表示可通过网络组合间接推导相对关系。
- relative bias 可直接加到 attention logits。
- RoPE 则通过旋转 Q/K，使点积依赖相对位置。

常见失分点 把“relative 更好”绝对化；任务、长度与实现都影响效果。

继续追问 相对位置是否还需要知道序列起点？

一语中的： Absolute position 表示 token 自身的坐标 p_i ；relative position 直接建模 $i-j$ 或距离/方向。

Q029 RoPE 的核心数学是什么？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 高级 Transformer 面试必考：要能从旋转矩阵推导相对位置。

30 秒答案

RoPE 对 Q/K 的二维特征对施加随位置变化的旋转。 $q_m = R_m q$, $k_n = R_n k$, 则 $q_m^T k_n = q^T R_m^T R_n k = q^T R_{\{n-m\}} k$, 因此点积自然只依赖相对位移 $n-m$ 。

$$(R_m q)^T (R_n k) = q^T R_{\{n-m\}} k$$

深度拆解

- 绝对位置 m/n 通过旋转进入 Q/K。
- 点积中的旋转差转化成相对位置。
- 不同维度使用不同角频率。
- RoPE 不需要把一个 position vector 直接加到 token embedding。

常见失分点 只回答“RoPE 是旋转位置编码”，无法推导 $R_m^T R_n$ 。

继续追问 二维旋转为什么保持向量范数？这对点积尺度有什么好处？

一语中的： RoPE 对 Q/K 的二维特征对施加随位置变化的旋转。

Q030 为什么 RoPE 通常作用在 Q/K 而不是 V？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解位置应主要影响“匹配关系”还是“被读取内容”。

30 秒答案

Attention 的路由权重由 QK^T 决定，RoPE 把位置关系编码进 Q/K 后可直接改变“谁关注谁”；V 更接近实际被读取的信息载体，因此经典 RoPE 不需要旋转 V。

深度拆解

- 如果旋转 V，会把位置变换混入输出内容空间。
- 这不是数学上绝对禁止，而是 RoPE 的标准设计选择。
- GQA/MQA 中每个 KV head 的 K 同样需要正确应用 RoPE。

常见失分点 误以为 V 完全不包含位置信息；上游 hidden state 本身已经受到位置相关计算影响。

继续追问 KV Cache 场景下，新 token 的 RoPE position 应如何计算？

一语中的： Attention 的路由权重由 QK^T 决定，RoPE 把位置关系编码进 Q/K 后可直接改变“谁关注谁”；V 更接近实际被读取的信息载体，因此经典 RoPE 不需要旋转 V。

Q031 RoPE 为什么会遇到长上下文外推问题？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 从频率/相位角度解释 context extension。

30 秒答案

训练只覆盖有限 position range。推理到远大于训练长度时，各频率维度会进入训练未覆盖的旋转相位，尤其高频部分变化快，导致注意力几何分布外移，长距离 retrieval 和 perplexity 可能恶化。

深度拆解

- “代码支持 64K” 不等于模型有效利用 64K。
- 低频维度对长距离更缓慢，高频维度对局部位置敏感。
- 扩展方法通常对位置或频率进行缩放/插值，并配合继续训练。

常见失分点 说 “RoPE 天然无限外推，所以没有长度限制”。

继续追问 为什么只做 position scaling 而完全不继续训练，质量仍可能下降？

一语中的：训练只覆盖有限 position range。

Q032 ALiBi 与 RoPE 的思想有什么区别？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 比较 “改变表示几何” 和 “改变注意力偏置”。

30 秒答案

ALiBi 直接在 attention logits 上加入与距离相关的线性 bias，通常让更远位置默认受到更大惩罚；RoPE 则旋转 Q/K，使点积结构包含相对位置。二者注入位置的位置和归纳偏置不同。

$$\text{score}_{ij} = q_i^T k_j - m_h \cdot |i-j|$$

深度拆解

- ALiBi 不需要显式位置 embedding table。
- 不同 head 可使用不同 slope。
- RoPE 的位置影响与内容向量点积耦合更深。

常见失分点 把 ALiBi 说成 “另一种位置 embedding 向量”。

继续追问 为什么不同 head 使用不同距离 slope 可能有意义？

一语中的：ALiBi 直接在 attention logits 上加入与距离相关的线性 bias，通常让更远位置默认受到更大惩罚；RoPE 则旋转 Q/K，使点积结构包含相对位置。

Q033 Position Interpolation 的基本思想是什么？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解长上下文扩展中的 “把新位置映回训练域”。

30 秒答案

若训练最大长度 T_{train} ，希望推理到 T_{new} ，可把新位置 p 映射为 $p \cdot T_{\text{train}} / T_{\text{new}}$ ，使位置相位仍落在较熟悉的训练范围内。它牺牲局部位置分辨率来换取更大的有效长度。

$$p' = p \cdot T_{\text{train}} / T_{\text{new}}$$

深度拆解

- 本质是 position compression。
- 越大扩展倍率，原本相邻 token 的有效位置差越小。
- 实际方法常配合 fine-tuning 与频率分段策略。

常见失分点 把插值说成“增加更多 position embedding”。

继续追问 如果扩展 8 倍，局部 token 的相对角度会发生什么？

一语中的：若训练最大长度 T_{train} ，希望推理到 T_{new} ，可把新位置 p 映射为 $p \cdot T_{train}/T_{new}$ ，使位置相位仍落在较熟悉的训练范围内。

Q034 技术上支持 64K 与真正有效 64K 有什么区别？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察评测意识：上下文窗口是能力而非 config 参数。

30 秒答案

Technical context window 表示模型和运行时能接受这么多 token；**effective context window** 表示模型能在远距离上可靠检索、组合和利用信息。两者应通过长上下文 perplexity、needle retrieval、多跳 QA、position sensitivity 等实验区分。

深度拆解

- KV Cache 能装下 $\neq$ attention 能正确利用。
- 训练数据里缺乏真实长依赖时，模型可能只使用近邻。
- 评测要覆盖不同 needle 位置、不同干扰强度和跨段推理。

常见失分点 用“最大 context length”直接代表模型长上下文能力。

继续追问 如何设计一个避免“关键词匹配投机”的长上下文评测？

一语中的：Technical context window 表示模型和运行时能接受这么多 token；effective context window 表示模型能在远距离上可靠检索、组合和利用信息。

Q035 Pre-LN 与 Post-LN 有什么区别？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察深层 Transformer 优化稳定性。

30 秒答案

Post-LN 将 Norm 放在 residual addition 之后；**Pre-LN** 先 Norm 再进入子层，residual 主路径更接近恒等映射。现代大模型常偏向 Pre-Norm，因为深层训练通常更稳定。

$$\text{Post: } \text{LN}(x + F(x)) \quad | \quad \text{Pre: } x + F(\text{LN}(x))$$

深度拆解

- Post-LN: $x_{l+1} = \text{LN}(x_l + F(x_l))$ 。
- Pre-LN: $x_{l+1} = x_l + F(\text{LN}(x_l))$ 。
- Pre-LN 并不是“效果必然更好”，而是优化性质更友好。
- 原始 Transformer 与现代 LLM 的 Norm placement 不完全相同。

常见失分点 把两者公式写反。

继续追问 为什么 Pre-LN 的 residual path 对梯度传播更直接？

一语中的： Post-LN 将 Norm 放在 residual addition 之后；Pre-LN 先 Norm 再进入子层，residual 主路径更接近恒等映射。

第四章 Norm、Residual 与 FFN

理解深层 Transformer 为什么能训练，以及 RMSNorm、SwiGLU 等现代模块为什么出现。

题号	题目	难度
Q036	为什么现代 LLM 普遍偏向 Pre-Norm?	4/5
Q037	LayerNorm 到底沿哪个维度计算?	2/5
Q038	RMSNorm 与 LayerNorm 有什么区别?	3/5
Q039	FFN 为什么不可缺少?	2/5
Q040	为什么经典 Transformer 常用 $d_{ff} \approx 4d_{model}$?	3/5
Q041	GELU 为什么比 ReLU 常见?	2/5
Q042	SwiGLU 是什么，为什么现代 LLM 常用?	3/5
Q043	为什么深层 Transformer 对初始化特别敏感?	4/5
Q044	为什么 Input Embedding 与 LM Head 经常 Weight Tying?	3/5
Q045	Encoder-only、Decoder-only、Encoder-Decoder 的本质区别?	2/5

Q036 为什么现代 LLM 普遍偏向 Pre-Norm?

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 需要把初始化梯度、warmup 与 identity path 联系起来。

30 秒答案

Pre-Norm 将归一化限制在 residual branch 的输入，主 residual stream 不被每层 Norm 包裹，形成更直接的梯度通路。相关分析表明 Post-LN 初始化时靠近输出层的梯度尺度可能更不稳定，因此对 warmup 更敏感。

深度拆解

- Pre-Norm 缓解的是优化稳定性，不代表完全不需要 warmup。
- 深层模型仍要控制 residual branch 输出尺度。
- DeepNorm 等方案说明 Post-LN 经过重新缩放也可以训练很深。

常见失分点 回答“因为 Pre-LN 梯度永远不会消失”。

继续追问 Pre-LN 深度很大时也可能有什么表示/尺度问题？

一语中的：Pre-Norm 将归一化限制在 residual branch 的输入，主 residual stream 不被每层 Norm 包裹，形成更直接的梯度通路。

Q037 LayerNorm 到底沿哪个维度计算？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 代码题常见 trap。

30 秒答案

对 [B,T,d] 的 Transformer hidden state，标准 LayerNorm 通常对最后的 d 维独立计算均值和方差，因此每个 batch、每个 token 都有自己的统计量。

深度拆解

- normalized_shape 常写成 d_model。
- 不会跨 token 混合统计。
- 这与 BatchNorm 跨 batch/空间维统计完全不同。

常见失分点 沿 T 维计算；或者把 padding token 混入同一统计量。

继续追问 如果对 [B,T,d] 错误地沿 T 维 Norm，会改变什么归纳偏置？

一语中的：对 [B,T,d] 的 Transformer hidden state，标准 LayerNorm 通常对最后的 d 维独立计算均值和方差，因此每个 batch、每个 token 都有自己的统计量。

Q038 RMSNorm 与 LayerNorm 有什么区别？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解现代 LLM 常见归一化的简化。

30 秒答案

RMSNorm 只按均方根缩放，不减去均值；LayerNorm 同时做 mean-centering 和 variance normalization。RMSNorm 运算更简单，实践中在很多 LLM 上效果良好。

$$\text{RMSNorm}(x) = \gamma \cdot x / \sqrt{(\text{mean}(x^2) + \epsilon)}$$

深度拆解

- $\text{RMS}(x) = \sqrt{(\text{mean}(x^2) + \epsilon)}$ 。
- $\text{RMSNorm}(x) = \gamma \cdot x / \text{RMS}(x)$ 。
- 没有 β /mean-centering 的具体实现视模型而定。
- 它仍然是逐 token hidden-dim 归一化。

常见失分点 说 RMSNorm “不归一化方差”。它仍控制向量 RMS 尺度。

继续追问 如果输入整体加一个常数偏移，LayerNorm 与 RMSNorm 的响应有何不同？

一语中的：RMSNorm 只按均方根缩放，不减去均值；LayerNorm 同时做 mean-centering 和 variance normalization。

Q039 FFN 为什么不可缺少？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 从函数表达能力理解 Transformer block。

30 秒答案

Attention 主要把已有 value 在 token 间重组；FFN 对每个 token 做强非线性特征变换和扩维/压缩。两者交替让模型既能通信又能计算。

深度拆解

- 经典 FFN: $W_2 \phi(W_1 x)$ 。
- 通常 d_{ff} 显著大于 d_{model} 。
- 参数量的大头常在 MLP。
- gated FFN 进一步加入数据依赖门控。

常见失分点 把 FFN 当作“只是增加参数”。

继续追问 为什么所有 token 共享同一 FFN 权重仍然能处理不同语义？

一语中的： Attention 主要把已有 value 在 token 间重组；FFN 对每个 token 做强非线性特征变换和扩维/压缩。

Q040 为什么经典 Transformer 常用 $d_{ff} \approx 4d_{model}$?

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察容量/计算经验值与参数估算。

30 秒答案

4d 是原始 Transformer 的经典经验配置，不是理论定律。两层 FFN 参数量约 $2 \cdot d \cdot d_{ff}$ ；当 $d_{ff}=4d$ 时约为 $8d^2$ ，通常比 Attention 的约 $4d^2$ 更大。

$$\text{Params(FFN)} \approx 2d \cdot d_{ff}$$

深度拆解

- 现代 gated MLP 为保持相近参数/FLOPs，hidden ratio 常会调整。
- 不同模型 family 会选择不同 expansion ratio。
- 需要从总参数和 FLOPs 预算而不是死记 4 倍。

常见失分点 认为 4d 是数学证明得到的最优值。

继续追问 若改用 SwiGLU（三个投影），如何调整 hidden size 以控制参数量？

一语中的： 4d 是原始 Transformer 的经典经验配置，不是理论定律。

Q041 GELU 为什么比 ReLU 常见？

难度 2/5 | 中频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解平滑非线性与 Transformer recipe。

30 秒答案

GELU 不是简单 hard threshold，而是对输入按其大小平滑门控；BERT 等模型广泛采用。现代 decoder LLM 又常进一步采用 SiLU/SwiGLU。

深度拆解

- ReLU 在 0 处硬截断。
- GELU 对负值并非全部归零。
- 实际选择更依赖完整训练 recipe，而不是单点理论优势。

常见失分点 把 GELU 说成“可以防止梯度消失”这种泛化口号。

继续追问 GELU 与 SiLU 在形状和计算上有哪些相似与不同？

一语中的： GELU 不是简单 hard threshold，而是对输入按其大小平滑门控；BERT 等模型广泛采用。

Q042 SwiGLU 是什么，为什么现代 LLM 常用？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察 gated FFN。

30 秒答案

SwiGLU 使用一条 SiLU 激活分支与另一条线性分支逐元素相乘，再投影回 hidden size。门控使 FFN 的特征选择更具输入依赖性。

$$\text{FFN}(\mathbf{x}) = [\text{SiLU}(\mathbf{x}\mathbf{W}_1) \odot (\mathbf{x}\mathbf{W}_2)]\mathbf{W}_3$$

深度拆解

- 典型形式: $[\text{SiLU}(\mathbf{x}\mathbf{W}_1) \odot (\mathbf{x}\mathbf{W}_2)]\mathbf{W}_3$ 。
- 相较普通两投影 FFN，多一个输入投影，因此常调整中间维度控制参数。
- 它改善的是 MLP 表达/训练质量，不是 Attention。

常见失分点 把 SwiGLU 写成 $\text{SiLU}(\mathbf{W}\mathbf{x})$ 后直接线性，没有 gate。

继续追问 为什么 gate 乘法能增加函数表达灵活性？

一语中的： SwiGLU 使用一条 SiLU 激活分支与另一条线性分支逐元素相乘，再投影回 hidden size。

Q043 为什么深层 Transformer 对初始化特别敏感？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解 residual 累积与 activation/gradient 尺度。

30 秒答案

L 层 residual 叠加可粗略看成 $\mathbf{x}_L = \mathbf{x}_0 + \Sigma \mathbf{F}_L$ 。若每个分支初始输出尺度过大，variance 会随深度积累，造成 activation/gradient 不稳。因此初始化、Norm placement、residual scaling 需要协同设计。

深度拆解

- 并非有 LayerNorm 就能忽略初始化。
- 特别深模型会使用 scaled init、DeepNorm、 μP 等思想。
- 训练稳定性问题往往在前几百步最容易暴露。

常见失分点 只背 Xavier/He 初始化名字，不分析 residual stack。

继续追问 如果 residual branch 的输出方差与层数无关地保持常数， L 很大时总 variance 会如何变化？

一语中的： L 层 residual 叠加可粗略看成 $x_L = x_0 + \Sigma F_l$ 。

Q044 为什么 Input Embedding 与 LM Head 经常 Weight Tying?

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察词表参数、表示空间与模型大小。

30 秒答案

输入 embedding $E \in \mathbb{R}^{V \times d}$ 与输出 LM head 可共享，使 $\text{logits} = hE^T$ 。这样减少约 $V \cdot d$ 级别参数，并让输入/输出 token 使用同一个语义基底。

深度拆解

- 词表 V 很大时节省明显。
- 是否共享取决于 architecture 与 tokenizer。
- 共享不意味着输入 embedding 与最终 hidden state 完全对称。

常见失分点 说“共享后参数减少一半”；实际只减少相应 embedding/head 一份，整体比例取决于模型大小。

继续追问 如果 hidden size 与 embedding size 不同，如何实现 weight tying?

一语中的： 输入 embedding $E \in \mathbb{R}^{V \times d}$ 与输出 LM head 可共享，使 $\text{logits} = hE^T$ 。

Q045 Encoder-only、Decoder-only、Encoder-Decoder 的本质区别？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 从 attention mask 与信息流解释三种范式。

30 秒答案

Encoder-only 使用双向 self-attention，适合表示学习；Decoder-only 使用 causal self-attention，适合自回归生成；Encoder-Decoder 用双向 Encoder 编码输入，再由 causal Decoder 通过 cross-attention 条件生成输出。

深度拆解

- 结构差异与预训练目标通常相互匹配。
- 任务并非绝对绑定：Decoder-only 也可分类，Encoder 也可做 span prediction。
- 现代通用 LLM 选择 Decoder-only 的重要原因之一是目标与接口统一。

常见失分点 用“BERT/GPT/T5”三个名字代替结构解释。

继续追问 为什么 retrieval encoder 更常用双向 Encoder?

一语中的: Encoder-only 使用双向 self-attention, 适合表示学习; Decoder-only 使用 causal self-attention, 适合自回归生成; Encoder-Decoder 用双向 Encoder 编码输入, 再由 causal Decoder 通过 cross-attention 条件生成输出。

第五章 BERT、GPT 与 Encoder-Decoder 范式

把预训练目标、Mask 结构与任务形态联系起来，而不是背模型名字。

题号	题目	难度
Q046	BERT 为什么可以双向？	2/5
Q047	BERT 为什么用 MLM，而 GPT 用 CLM？	3/5
Q048	BERT 的 NSP 是什么，为什么后来常被移除？	3/5
Q049	GPT 的 Causal LM 训练目标是什么？	2/5
Q050	为什么自回归模型训练时还能并行？	2/5
Q051	为什么 Decoder-only 成为通用 LLM 主流之一？	3/5
Q052	Cross-Attention 为什么适合机器翻译？	3/5
Q053	Teacher Forcing 是什么？	3/5
Q054	Exposure Bias 是什么？	3/5
Q055	Decoder LM 的 label 为什么必须 shift？	2/5

Q046 BERT 为什么可以双向？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解双向上下文与 masking objective。

30 秒答案

BERT Encoder 没有 causal mask，位置 i 可以同时读取左、右上下文，因此每层表示都联合条件于两侧 token。为避免直接看到被预测 token，本质训练目标采用 MLM。

深度拆解

- 双向指的是 attention visibility，不是两个 RNN 方向拼接。
- 预训练时被 mask 的 token 需要用上下文恢复。
- 下游 fine-tuning 时通常不再随机 mask 输入。

常见失分点 说“因为 BERT 同时从左到右和从右到左训练两个模型”。

继续追问 为什么直接 next-token prediction 无法在同一位置使用未来 token？

一语中的： BERT Encoder 没有 causal mask，位置 i 可以同时读取左、右上下文，因此每层表示都联合条件于两侧 token。

Q047 BERT 为什么用 MLM，而 GPT 用 CLM？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 把目标函数与可见上下文联系起来。

30 秒答案

CLM 通过 causal factorization 预测下一个 token，天然适合生成；BERT 想让每个位置看到左右上下文，因此必须遮挡部分 token 再恢复，否则目标 token 已经在输入中。

深度拆解

- MLM 只对被选中 token 计算预测 loss。
- BERT 原始做法对约 15% token 选择预测。
- MLM 与实际 fine-tuning 输入存在 [MASK] distribution mismatch，后续工作对此有不同改进。

常见失分点 说 MLM “比 CLM 更高级”。

继续追问 为什么 MLM 对生成任务不如 causal LM 直接？

一语中的： CLM 通过 causal factorization 预测下一个 token，天然适合生成；BERT 想让每个位置看到左右上下文，因此必须遮挡部分 token 再恢复，否则目标 token 已经在输入中。

Q048 BERT 的 NSP 是什么，为什么后来常被移除？

难度 3/5 | 中频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察是否理解预训练目标的消融结论。

30 秒答案

NSP 是判断两个 segment 是否真实相邻的二分类辅助任务，初衷是帮助句间关系建模。后续如 RoBERTa 的实验表明，在更强数据与训练 recipe 下去掉 NSP 也能获得更好结果，因此它不是 BERT 成功的必要核心。

深度拆解

- NSP negative 往往来自随机文档，可能过于容易。
- 更重要的是数据规模、dynamic masking、训练步数与 batch 等整体 recipe。
- 不要把“被移除”误读为所有句间目标都无用。

常见失分点 把 NSP 说成“永远没用”。

继续追问 如何设计更难的 sentence-order / discourse objective？

一语中的： NSP 是判断两个 segment 是否真实相邻的二分类辅助任务，初衷是帮助句间关系建模。

Q049 GPT 的 Causal LM 训练目标是什么？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 必须能写出概率分解与 loss。

30 秒答案

自回归语言模型把联合概率分解为 $\prod_t P(x_t | x_{<t})$ ，训练用 next-token cross-entropy。Causal mask 保证每个位置只依赖过去。

$$P(x_1:T) = \prod_t P(x_t | x_{<t})$$

深度拆解

- 训练 labels 通常相对 inputs shift 一位。
- 整段 token 可并行 forward，因 ground-truth prefix 已知。
- 推理时未来 token 未知，因此必须逐步采样。

常见失分点 把“自回归”理解成训练也必须 for-loop 每个 token。

继续追问 为什么 teacher forcing 能并行但 sampling 不能并行？

一语中的：自回归语言模型把联合概率分解为 $\prod_t P(x_t | x_{<t})$ ，训练用 next-token cross-entropy。

Q050 为什么自回归模型训练时还能并行？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 这是非常高频的“反直觉”题。

30 秒答案

训练样本的完整序列已知，可以一次输入全部 token；causal mask 在计算图里屏蔽未来信息，因此所有位置的 next-token logits 可同时算出。生成时后续 token 尚未存在，必须等前一步结果。

深度拆解

- 并行的是同一序列各位置的 forward。
- 依赖关系由 mask 保证，而不是物理执行顺序。
- 这也是 Transformer 相比 RNN 训练吞吐高的重要来源。

常见失分点 回答“因为 GPU 能并行”，没有说明 ground truth 已知。

继续追问 如果做 scheduled sampling，训练并行性会发生什么变化？

一语中的：训练样本的完整序列已知，可以一次输入全部 token；causal mask 在计算图里屏蔽未来信息，因此所有位置的 next-token logits 可同时算出。

Q051 为什么 Decoder-only 成为通用 LLM 主流之一？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 从目标统一、数据接口与 scaling 解释。

30 秒答案

Decoder-only 用一个 next-token objective 就能把文档、对话、代码、工具轨迹等统一为 token stream；结构简单、scaling 与 serving 统一、in-context learning 自然。但这并不意味着 Encoder 或 Encoder-Decoder 在所有任务上都落后。

深度拆解

- 统一接口降低多任务架构复杂度。
- prompt 和 answer 同一上下文中自然条件化。
- retrieval encoder、翻译/语音等场景仍可能更适合其他范式。

常见失分点 把主流归因成“Decoder 参数更多”。

继续追问 在固定总参数下，Encoder-Decoder 与 Decoder-only 的算力分配有什么不同？

一语中的：Decoder-only 用一个 next-token objective 就能把文档、对话、代码、工具轨迹等统一为 token stream；结构简单、scaling 与 serving 统一、in-context learning 自然。

Q052 Cross-Attention 为什么适合机器翻译？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 从 soft alignment 和 conditional generation 解释。

30 秒答案

Decoder 当前 query 可以对 Encoder 的所有 source token 动态分配权重，从而实现“生成当前目标 token 时应该读输入的哪部分”。这相当于可学习的 soft alignment。

Cross score shape: $[T_target \times T_source]$

深度拆解

- Encoder memory 在生成过程中固定，可缓存 K/V。
- 不同 decoder position 可选择不同 source 区域。
- 语言顺序不一致时这种动态对齐尤其重要。

常见失分点 只说“可以看输入”。

继续追问 如果源序列 4K、目标序列 1K，Cross-Attention 的复杂度如何估算？

一语中的：Decoder 当前 query 可以对 Encoder 的所有 source token 动态分配权重，从而实现“生成当前目标 token 时应该读输入的哪部分”。

Q053 Teacher Forcing 是什么？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解训练与推理分布差异。

30 秒答案

训练 decoder 时使用真实历史 token $y_{<t}$ 作为条件，而不是模型自己生成的历史 $\hat{y}_{<t}$ 。这样训练稳定且可并行，但会造成 inference 时的输入分布 mismatch。

深度拆解

- ground-truth prefix 提供干净条件。

- 推理时任何早期错误会进入后续上下文。
- 并非所有现代 LM 训练都用“teacher forcing”这个词，但 next-token MLE 本质上使用真实 prefix。

常见失分点 认为 teacher forcing 是“把答案直接放进输入导致泄漏”；causal shift 后仍是正确条件。

继续追问 Scheduled sampling 想解决什么，为什么实践不一定简单？

一语中的：训练 decoder 时使用真实历史 token $y_{<t}$ 作为条件，而不是模型自己生成的历史 $\hat{y}_{<t}$ 。

Q054 Exposure Bias 是什么？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 要求清楚区分 MLE 的真实 prefix 与推理的模型 prefix。

30 秒答案

训练优化的是 $P(y_t|y_{<t}^{true})$ ，而推理面对 $P(y_t|\hat{y}_{<t})$ 。若模型前面犯错，后续进入训练较少见的 prefix，错误可能累积。这种 train-test prefix distribution mismatch 常称 exposure bias。

深度拆解

- 它不是“模型过拟合训练集”的同义词。
- RL、sequence-level objective、data augmentation 等可从不同角度缓解。
- 大型 LLM 仍以 next-token MLE 为核心，说明 exposure bias 是 trade-off 而非简单致命缺陷。

常见失分点 把 exposure bias 与 label leakage 混淆。

继续追问 Beam Search 能否消除 exposure bias？为什么不能？

一语中的：训练优化的是 $P(y_t|y_{<t}^{true})$ ，而推理面对 $P(y_t|\hat{y}_{<t})$ 。

Q055 Decoder LM 的 label 为什么必须 shift？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 Coding 面试基础，直接决定训练目标是否正确。

30 秒答案

输入位置 t 的 hidden state 应预测下一个 token x_{t+1} ，因此常见做法是 logits[:, :-1] 对 labels[:, 1:]。若 label 与 input 同位置，模型可能学成错误的 identity-like objective。

$$\text{input: } x_0 \dots x_{T-2} \rightarrow \text{labels: } x_1 \dots x_{T-1}$$

深度拆解

- 许多 Hugging Face causal LM model 内部会自动 shift，重复手动 shift 反而出错。
- padding/EOS 的 loss mask 要与 shift 后位置对齐。
- packed sequence 还需处理不同样本边界。

常见失分点 在模型已自动 shift 的 API 外再 shift 一次。

继续追问 如何用一个 4-token toy sequence 人工验证 loss alignment?

一语中的: 输入位置 t 的 hidden state 应预测下一个 token x_{t+1} , 因此常见做法是 logits[:, :-1] 对 labels[:, 1:]。

第六章 训练、优化与 Debug

覆盖 label shift、NaN、AdamW、warmup、混合精度、梯度与分布式训练。

题号	题目	难度
Q056	Padding token 为什么不能参与 loss?	3/5
Q057	Transformer loss 变成 NaN，如何系统排查?	4/5
Q058	为什么训练早期常用 Learning Rate Warmup?	3/5
Q059	Adam 与 AdamW 的核心区别?	3/5
Q060	Gradient Clipping 为什么有用?	3/5
Q061	Gradient Accumulation 是否完全等价于大 Batch?	4/5
Q062	FP16 与 BF16 为什么训练表现不同?	3/5
Q063	训练正常但验证每次结果不同，先查什么?	2/5
Q064	Transformer 初始化不合理有哪些症状?	4/5
Q065	固定训练 FLOPs，模型更大还是 token 更多?	4/5
Q066	Transformer 多 GPU 有哪些并行方式?	4/5
Q067	Decoder-only LLM 的生成过程是什么?	2/5

Q056 Padding token 为什么不能参与 loss?

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察真实 batch 的目标 mask。

30 秒答案

padding 只是为了 batch 对齐，不是语言模型应学习的真实目标。若大量 pad 参与 CE，模型可通过预测 pad 获得虚假的低 loss，并扭曲梯度。

深度拆解

- 使用 ignore_index 或 loss mask。
- Attention mask 与 loss mask 是两个不同层面。
- packed sequence 可减少 padding，但会引入 sample boundary mask 问题。

常见失分点 只在 attention 里 mask pad，却仍让 pad label 参与 loss。

继续追问 如果 tokenizer 把 pad_id 与 eos_id 设成同一个值，loss mask 应如何设计？

一语中的：padding 只是为了 batch 对齐，不是语言模型应学习的真实目标。

Q057 Transformer loss 变成 NaN，如何系统排查？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 真正的工程面试题：看排障顺序与可观测性。

30 秒答案

先排数据与 mask，再看 forward 中 logits/attention、再看 loss、gradient norm 和 optimizer/AMP。不要第一反应只调小 learning rate。

深度拆解

- 数据：token id、长度、空样本、异常值。
- mask：是否存在整行全 $-\infty$ ，broadcast 是否正确。
- forward：逐层记录 min/max/mean/std/isnan/isinf。
- backward：定位第一个出现 NaN/Inf 的 gradient。
- optimizer：LR、weight decay、loss scaling、step 顺序。

常见失分点 一看到 NaN 就加 gradient clipping；这可能掩盖真正 bug。

继续追问 如何用 autograd anomaly detection 或 hook 定位第一处 NaN？

一语中的：先排数据与 mask，再看 forward 中 logits/attention、再看 loss、gradient norm 和 optimizer/AMP。

Q058 为什么训练早期常用 Learning Rate Warmup？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解优化器状态与深层 Transformer 的早期不稳定。

30 秒答案

训练初期参数表示与 Adam moments 尚未形成稳定尺度，大 learning rate 容易造成过大的有效更新。Warmup 先从小 LR 增长到峰值，再进入衰减阶段，可改善早期稳定性。

深度拆解

- 原始 Transformer 就使用 warmup scheduler。
- Norm placement 与初始化会影响 warmup 敏感度。
- warmup steps 应结合总 steps、batch/token budget，而不是固定数字。

常见失分点 说“warmup 是防止过拟合”。

继续追问 如果 batch size 扩大 8 倍，LR 与 warmup 应如何重新评估？

一语中的：训练初期参数表示与 Adam moments 尚未形成稳定尺度，大 learning rate 容易造成过大的有效更新。

Q059 Adam 与 AdamW 的核心区别？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察 decoupled weight decay。

30 秒答案

AdamW 将 weight decay 直接作用到参数更新，与自适应梯度缩放解耦；而把 L2 正则项直接加到 gradient 中会被 Adam 的 per-parameter adaptive scaling 改变其实际效果。

$$\theta \leftarrow (1 - \eta \lambda) \theta - \eta \cdot \text{AdamUpdate}$$

深度拆解

- 现代 Transformer 训练常用 AdamW。
- bias 和 Norm 参数通常不做或少做 weight decay。
- weight decay 与 learning rate 共同决定每步参数收缩。

常见失分点 把 AdamW 理解成 “Adam + 更大的 weight decay”。

继续追问 为什么 LayerNorm scale/bias 常从 weight decay 中排除？

一语中的： AdamW 将 weight decay 直接作用到参数更新，与自适应梯度缩放解耦；而把 L2 正则项直接加到 gradient 中会被 Adam 的 per-parameter adaptive scaling 改变其实际效果。

Q060 Gradient Clipping 为什么有用？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解极端 batch 与更新稳定性。

30 秒答案

Global norm clipping 在梯度范数超过阈值 c 时按比例缩放全部梯度，使方向不变而控制单步更新幅度。它是保险机制，不是修复坏数据或错误 LR 的万能药。

$$g \leftarrow g \cdot \min(1, c / \|g\|)$$

深度拆解

- 通常在 optimizer.step 前执行。
- AMP 下要先 unscale 再 clip，避免阈值作用在缩放梯度上。
- 监控 clipping 触发比例很重要；如果几乎每步都触发，说明设置/训练可能异常。

常见失分点 在 scaler.unscale_ 前用真实阈值 clip FP16 scaled gradient。

继续追问 per-parameter clipping 与 global norm clipping 有什么不同？

一语中的： Global norm clipping 在梯度范数超过阈值 c 时按比例缩放全部梯度，使方向不变而控制单步更新幅度。

Q061 Gradient Accumulation 是否完全等价于大 Batch?

难度 4/5 | 高频 | 面试形式: 概念 / 推导 / 追问

面试官在考什么 考察 optimizer/scheduler、随机性与归一化细节。

30 秒答案

在 loss 正确归一化、只在累计 K 个 micro-batch 后 step 的理想条件下, 梯度和接近一次大 batch; 但 dropout 随机性、clip 时机、scheduler step、BatchNorm 等因素都可能造成差异。

深度拆解

- $\text{effective batch} = B_{\text{micro}} \times K \times \text{data_parallel_world_size}$ 。
- 每个 micro-batch loss 若都求 mean, 需要确认累计时是否再除 K。
- scheduler 通常按 optimizer step 而不是 micro-step 更新。

常见失分点 累计 K 次后忘记 loss/K, 导致梯度放大 K 倍。

继续追问 数据并行 AllReduce 应每 micro-step 做还是只最后一步做? 如何用 no_sync 优化?

一语中的: 在 loss 正确归一化、只在累计 K 个 micro-batch 后 step 的理想条件下, 梯度和接近一次大 batch; 但 dropout 随机性、clip 时机、scheduler step、BatchNorm 等因素都可能造成差异。

Q062 FP16 与 BF16 为什么训练表现不同?

难度 3/5 | 高频 | 面试形式: 概念 / 推导 / 追问

面试官在考什么 理解浮点格式的 exponent range 与 loss scaling。

30 秒答案

FP16 的指数范围较窄, 更容易 overflow/underflow; BF16 保留与 FP32 类似的 exponent 范围但 mantissa 更短, 因此动态范围大、训练大模型更稳, 通常更少依赖 loss scaling。

深度拆解

- BF16 精度并非处处高于 FP16; 它是范围更大。
- 某些关键 reduction/optimizer state 仍保留 FP32。
- 硬件是否原生支持 BF16 会影响性能。

常见失分点 说“BF16 比 FP16 更精确”。

继续追问 为什么 Adam moments 常用 FP32 保存?

一语中的: FP16 的指数范围较窄, 更容易 overflow/underflow; BF16 保留与 FP32 类似的 exponent 范围但 mantissa 更短, 因此动态范围大、训练大模型更稳, 通常更少依赖 loss scaling。

Q063 训练正常但验证每次结果不同, 先查什么?

难度 2/5 | 高频 | 面试形式: 概念 / 推导 / 追问

面试官在考什么 考察最基本的 train/eval 状态与随机性排查。

30 秒答案

先确认 `model.eval()`，使 dropout 等随机层进入推理模式；再检查随机采样、seed、data order、非确定性 kernel 与分布式 sampler。

深度拆解

- 如果使用生成评测，temperature/top-p 也会引入随机性。
- `eval()` 不会自动关闭梯度，通常还配合 `no_grad/inference_mode`。
- 某些模型使用 stochastic depth 或 router noise，也需确认评估配置。

常见失分点 只设置 `torch.manual_seed` 就以为完全确定。

继续追问 为何 CUDA 某些 kernel 即使固定 seed 仍可能非确定？

一语中的：先确认 `model.eval()`，使 dropout 等随机层进入推理模式；再检查随机采样、seed、data order、非确定性 kernel 与分布式 sampler。

Q064 Transformer 初始化不合理有哪些症状？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 从训练曲线与统计量反推初始化问题。

30 秒答案

可能出现 activation std 随层爆炸/衰减、attention logits 极端、前几步 loss 抖动、gradient norm 异常、深层几乎不学习等。排查应查看按层 activation/gradient statistics。

深度拆解

- 不要只观察总 loss。
- Norm 能缓解但不能消除所有尺度问题。
- 深度、residual scale、LR 与初始化相互作用。

常见失分点 认为“loss 能下降就说明初始化没问题”。

继续追问 如何用 1 个 forward 的 layer-wise std 曲线判断潜在尺度异常？

一语中的：可能出现 activation std 随层爆炸/衰减、attention logits 极端、前几步 loss 抖动、gradient norm 异常、深层几乎不学习等。

Q065 固定训练 FLOPs，模型更大还是 token 更多？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察 compute-optimal scaling，而非单参数崇拜。

30 秒答案

模型参数与训练 token 是耦合的。固定 compute 下，参数过大但 token 太少会欠训练；参数太小、数据过多也可能浪费 capacity。应根据 scaling law、目标任务、数据质量与 serving 约束共同选择。

深度拆解

- Chinchilla 类结论强调许多早期大模型相对 data-undertrained。
- “最佳比例”不是跨模型/数据永恒常数。
- 高质量 token 与重复 epoch 也会改变有效数据量。

常见失分点 机械背 “20 token/parameter” 并当成普遍定律。

继续追问 如果高质量 unique data 只有 1T token，但 compute-optimal 需要 4T，怎么办？

一语中的：模型参数与训练 token 是耦合的。

Q066 Transformer 多 GPU 有哪些并行方式？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 判断是否具备大模型训练系统视角。

30 秒答案

常见有 Data Parallel、Tensor Parallel、Pipeline Parallel、Sequence/Context Parallel，以及 ZeRO/FSDP 类参数/梯度/优化器状态分片。大模型通常组合多维并行。

深度拆解

- DP：复制模型、切 batch、聚合梯度。
- TP：切单层大矩阵/attention heads。
- PP：切层并用 micro-batch 填 pipeline。
- CP/SP：沿 sequence/token 维切分相关张量。
- 选择方案要看模型大小、序列长度、网络拓扑和通信/显存瓶颈。

常见失分点 把 FSDP 直接等同于 Tensor Parallel。

继续追问 为什么 TP 更依赖高速 NVLink，而 DP 可以跨更慢的节点网络扩展？

一语中的：常见有 Data Parallel、Tensor Parallel、Pipeline Parallel、Sequence/Context Parallel，以及 ZeRO/FSDP 类参数/梯度/优化器状态分片。

Q067 Decoder-only LLM 的生成过程是什么？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 确保真正理解 autoregressive dependency。

30 秒答案

给定 prefix $x_1:T$ ，模型预测 $P(x_{T+1}|x_{\leq T})$ ，采样/选择新 token 后把它追加，再预测下一步。输出 N 个 token 至少存在 N 个依赖的 decoding steps。

深度拆解

- prefill 可并行处理整个 prompt。
- decode 每步通常只新增 1 token。
- sampling policy 决定从 logits 到 token 的选择方式。

常见失分点 把模型一次 forward 输出的所有位置 logits 当成可以一次生成未来 N 个 token。

继续追问 为什么 speculative decoding 可以“一次推进多个 token”却仍需 target model 验证？

一语中的： 给定 prefix $x_1:T$ ，模型预测 $P(x_{T+1}|x_{\leq T})$ ，采样/选择新 token 后把它追加，再预测下一步。

第七章 自回归推理与 KV Cache

从增量解码推导 KV Cache、GQA/MQA、采样、TTFT/TPOT 与 serving 内存。

题号	题目	难度
Q068	KV Cache 缓存了什么，为什么不缓存 Q?	3/5
Q069	KV Cache 内存如何估算?	4/5
Q070	为什么训练不需要像 decode 一样使用 KV Cache?	3/5
Q071	Prefill 与 Decode 的瓶颈有什么不同?	4/5
Q072	MHA、MQA、GQA 的区别?	3/5
Q073	Greedy、Beam Search 与 Sampling 如何选择?	3/5
Q074	Temperature、Top-k、Top-p 分别做什么?	2/5
Q075	为什么模型会陷入重复生成?	3/5
Q076	LLM Serving 的关键 latency 指标有哪些?	3/5
Q077	Continuous Batching 为什么适合 LLM?	4/5
Q078	PagedAttention 解决什么问题?	4/5
Q079	Speculative Decoding 为什么能加速且保持目标分布?	4/5
Q080	FlashAttention 到底优化了什么?	3/5

Q068 KV Cache 缓存了什么，为什么不缓存 Q?

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 高级推理面试核心。

30 秒答案

过去 token 的 K/V 在后续自回归步骤中不会变化，因此缓存它们可避免重复投影和重复构建历史 memory；当前 query 每一步只在当前 token 上使用，过去 Q 不再需要参与新的 attention，因此通常不缓存 Q。

$$q_t [K_1...K_t]^T \rightarrow weights \rightarrow [V_1...V_t]$$

深度拆解

- 每层都有自己的 K/V cache。
- 新 token 只计算 k_new,v_new，再追加到 cache。
- RoPE/position index 必须和 cache length 对齐。
- Cross-attention 的 encoder K/V 也可缓存。

常见失分点 说“缓存 attention matrix”；标准 KV cache 缓存的是 key/value hidden projections。

继续追问 如果使用 sliding-window attention，KV Cache 可以如何裁剪？

一语中的：过去 token 的 K/V 在后续自回归步骤中不会变化，因此缓存它们可避免重复投影和重复构建历史 memory；当前 query 每一步只在当前 token 上使用，过去 Q 不再需要参与新的 attention，因此通常不缓存 Q。

Q069 KV Cache 内存如何估算？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 要求能现场算 serving capacity。

30 秒答案

若 L 层、batch B、上下文 T、KV heads H_{kv} 、head dim D_h 、每元素 s bytes，则 K 和 V 两份：
 $M \approx 2 \cdot L \cdot B \cdot T \cdot H_{kv} \cdot D_h \cdot s$ 。标准 MHA 的 $H_{kv}=H_q$ ，GQA/MQA 可显著下降。

$$M_{KV} = 2 \cdot L \cdot B \cdot T \cdot H_{kv} \cdot D_h \cdot \text{bytes}$$

深度拆解

- 注意单位换算 MiB/GiB。
- beam search 与多并发 request 会乘上额外 batch/beam 因子。
- 量化 KV cache 可进一步降低 s，但可能影响质量/Kernel。

常见失分点 忘记 K/V 两份或忘记 layers。

继续追问 给定 $L=32$ 、 $T=32K$ 、 $H_{kv}=8$ 、 $D_h=128$ 、BF16，单请求 KV 约多少 GiB？

一语中的：若 L 层、batch B、上下文 T、KV heads H_{kv} 、head dim D_h 、每元素 s bytes，则 K 和 V 两份：
 $M \approx 2 \cdot L \cdot B \cdot T \cdot H_{kv} \cdot D_h \cdot s$ 。

Q070 为什么训练不需要像 decode 一样使用 KV Cache？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 区分 full-sequence parallel training 与 incremental inference。

30 秒答案

训练时完整序列已知，一次矩阵乘即可得到所有位置的 Q/K/V 并做全序列 attention；KV Cache 的价值是避免自回归生成每一步重复计算历史 K/V。

深度拆解

- 训练 activation 本身会为了 backward 保存/重算，但那是不同概念。
- inference 无 backward，主要瓶颈转向 KV memory 与读带宽。
- prefix cache 则是跨 request 重用 prefill 结果。

常见失分点 把 activation checkpointing 与 KV caching 混为一谈。

继续追问 为什么训练长序列时仍可能做 sequence parallel，而不是 KV cache？

一语中的：训练时完整序列已知，一次矩阵乘即可得到所有位置的 Q/K/V 并做全序列 attention；KV Cache 的价值是避免自回归生成每一步重复计算历史 K/V。

Q071 Prefill 与 Decode 的瓶颈有什么不同？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 LLM serving 系统核心概念。

30 秒答案

Prefill 一次处理大量 prompt token，矩阵乘尺寸大、并行度高，通常更 compute-intensive；Decode 每步 token 数很少，却要读取越来越大的模型权重/KV cache，常更受 memory bandwidth 与调度影响。

深度拆解

- TTFT 主要受 queue+prefill 影响。
- TPOT/ITL 主要由 decode 性能决定。
- 短 prompt/长输出与长 prompt/短输出的优化重点不同。

常见失分点 只用总体 tokens/s 描述 latency。

继续追问 为什么 decode batch 增大可以提高吞吐，但同时恶化单请求 latency？

一语中的：Prefill 一次处理大量 prompt token，矩阵乘尺寸大、并行度高，通常更 compute-intensive；Decode 每步 token 数很少，却要读取越来越大的模型权重/KV cache，常更受 memory bandwidth 与调度影响。

Q072 MHA、MQA、GQA 的区别？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 直接连接模型质量与 KV Cache 内存。

30 秒答案

MHA 每个 Q head 有独立 K/V head；MQA 所有 Q heads 共享一套 K/V；GQA 让一组 Q heads 共享一个 KV head，是质量与推理效率的折中。

$$H_{kv}: MHA=H_q, GQA<H_q, MQA=1$$

深度拆解

- H_{kv} 从 H_q 降低会线性缩小 KV cache 的 head 维度部分。
- MQA 极端 $H_{kv}=1$ 。
- GQA 通常按 group 复制/映射 KV 到多个 Q heads。

常见失分点 认为 GQA 是把 query head 减少。

继续追问 从 pretrained MHA 转成 GQA 时，K/V 权重可以如何初始化/聚合？

一语中的：MHA 每个 Q head 有独立 K/V head；MQA 所有 Q heads 共享一套 K/V；GQA 让一组 Q heads 共享一个 KV head，是质量与推理效率的折中。

Q073 Greedy、Beam Search 与 Sampling 如何选择？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察 decoding objective 与任务差异。

30 秒答案

Greedy 每步取 argmax；Beam Search 保留多个高概率序列，适合翻译等较确定输出；开放式文本更常用 temperature/top-p 等 sampling，以避免最大似然路径过于保守或重复。

深度拆解

- Beam Search 增加 KV cache 与计算约 beam size 倍。
- 长度归一化会影响 beam 排名。
- sampling 的随机性可通过 seed 控制但仍依赖 kernel/并发。

常见失分点 认为 beam size 越大质量一定越好。

继续追问 为什么开放式对话中 beam search 常产生“安全但僵硬”的文本？

一语中的：Greedy 每步取 argmax；Beam Search 保留多个高概率序列，适合翻译等较确定输出；开放式文本更常用 temperature/top-p 等 sampling，以避免最大似然路径过于保守或重复。

Q074 Temperature、Top-k、Top-p 分别做什么？

难度 2/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 要从 logits/probability 分布解释。

30 秒答案

Temperature 用 z/T 调整分布尖锐度；Top-k 固定只保留概率最高的 k 个 token；Top-p 选择累计概率达到 p 的最小候选集合，因此候选数量随当前分布动态变化。

$$P_i = \text{softmax}(z_i/T)$$

深度拆解

- $T \rightarrow 0$ 接近 greedy（实现上不能真的除 0）。
- Top-p 在高置信状态候选少、平坦状态候选多。
- 通常先处理 logits，再归一化和采样，具体顺序取决于实现。

常见失分点 把 top-p 理解为“保留概率大于 p 的 token”。

继续追问 Top-k 与 Top-p 同时启用时，候选集如何组合？

一语中的：Temperature 用 z/T 调整分布尖锐度；Top-k 固定只保留概率最高的 k 个 token；Top-p 选择累计概率达到 p 的最小候选集合，因此候选数量随当前分布动态变化。

Q075 为什么模型会陷入重复生成？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 从 autoregressive feedback loop 解释。

30 秒答案

一旦生成进入某个高概率重复 pattern，重复 token/短语又成为下一步 prefix，使该 pattern 的后续概率进一步升高。训练分布、模型 calibration 和 decoding 策略都可能共同造成 repetition loop。

深度拆解

- 不是单纯“模型记忆差”。
- repetition penalty/n-gram blocking 可缓解但可能损害正常重复。
- 长输出中 exposure bias 与采样随机性会放大循环。

常见失分点 一律使用极强 repetition penalty，导致代码/列表等合法重复被破坏。

继续追问 如何区分是模型 logits 本身重复还是 sampling implementation bug？

一语中的：一旦生成进入某个高概率重复 pattern，重复 token/短语又成为下一步 prefix，使该 pattern 的后续概率进一步升高。

Q076 LLM Serving 的关键 latency 指标有哪些？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 系统面试必须区分用户体验与吞吐。

30 秒答案

至少区分 TTFT（首 token 延迟）、TPOT/ITL（后续 token 间延迟）、E2E latency、吞吐 tokens/s 或 req/s，以及 P50/P95/P99。

深度拆解

- TTFT 对长 prompt 与 queue 非常敏感。
- TPOT 对 decode batching、KV 带宽、模型并行敏感。
- 平均值可能掩盖尾延迟，线上更关注 P99。

常见失分点 只汇报“1000 tokens/s”却不说明 batch 与 latency。

继续追问 如何优化吞吐而不让 P99 TTFT 失控？

一语中的：至少区分 TTFT（首 token 延迟）、TPOT/ITL（后续 token 间延迟）、E2E latency、吞吐 tokens/s 或 req/s，以及 P50/P95/P99。

Q077 Continuous Batching 为什么适合 LLM?

难度 4/5 | 高频 | 面试形式: 概念 / 推导 / 追问

面试官在考什么 理解动态长度请求导致的资源碎片。

30 秒答案

不同请求的 prompt 和 generation 长度差异很大。Continuous batching 在每个 decoding iteration 动态加入新请求、移除完成请求，使 GPU batch 槽位持续被利用，比静态 batch 等最慢请求更高效。

深度拆解

- scheduler 需要在吞吐、TTFT、公平性之间折中。
- 不同 sequence length 会带来 KV memory 不同。
- chunked prefill 可进一步把长 prompt 与 decode 混合调度。

常见失分点 把 continuous batching 简化成 “batch size 每次变化”。

继续追问 为什么一个超长 prefill 请求可能阻塞大量 decode 请求？

一语中的: 不同请求的 prompt 和 generation 长度差异很大。

Q078 PagedAttention 解决什么问题?

难度 4/5 | 高频 | 面试形式: 概念 / 推导 / 追问

面试官在考什么 理解 KV Cache 的内存碎片与动态分配。

30 秒答案

KV Cache 随请求长度动态增长，如果要求每个请求占用一大段连续显存，会产生预留浪费和外部碎片。PagedAttention 将 KV 切成固定大小 blocks，用页表式映射让逻辑连续而物理可离散，提高显存利用率与并发。

深度拆解

- 核心不是改变 Attention 数学。
- block 可在请求结束后回收复用。
- beam/prefix sharing 可利用 copy-on-write 或 block sharing。

常见失分点 说 PagedAttention “压缩了 KV 数值”。

继续追问 block size 过大或过小分别会有什么问题？

一语中的: KV Cache 随请求长度动态增长，如果要求每个请求占用一大段连续显存，会产生预留浪费和外部碎片。

Q079 Speculative Decoding 为什么能加速且保持目标分布?

难度 4/5 | 高频 | 面试形式: 概念 / 推导 / 追问

面试官在考什么 理解 draft/target 两模型的并行验证。

30 秒答案

小 draft model 先提出多个候选 token，大 target model 用一次更大的 forward 并行验证；若连续接受多个候选，一次昂贵 target 调用就推进多个位置。严格算法用接受/校正规则保持与 target sampling 相同的目标分布。

深度拆解

- 加速取决于 draft 速度与接受率。
- draft 太弱接受率低，收益可能消失。
- 它减少的是 sequential target steps，而不是免费计算。

常见失分点 说“直接用小模型生成一半 token，所以质量会变差”。

继续追问 什么样的 draft model 才是好 draft：参数越小越好吗？

一语中的：小 draft model 先提出多个候选 token，大 target model 用一次更大的 forward 并行验证；若连续接受多个候选，一次昂贵 target 调用就推进多个位置。

Q080 FlashAttention 到底优化了什么？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 最常见高级误区题：复杂度 vs IO。

30 秒答案

FlashAttention 仍计算 exact Softmax Attention，数学 FLOPs 仍含二次项；它通过 tiling 与 online softmax 避免把完整 $T \times T$ score/probability 矩阵反复写入 HBM，大幅减少 HBM↔SRAM 的 IO。

Exact attention + fewer HBM reads/writes

深度拆解

- 它是 IO-aware kernel，而不是 approximation。
- 内存占用显著下降，使更长 sequence 可训练。
- wall-clock 速度来自减少昂贵内存访问和融合 kernel。

常见失分点 回答“把 $O(T^2)$ 变成 $O(T)$ ”。

继续追问 为什么理论 FLOPs 没变，实际速度仍能明显提升？

一语中的：FlashAttention 仍计算 exact Softmax Attention，数学 FLOPs 仍含二次项；它通过 tiling 与 online softmax 避免把完整 $T \times T$ score/probability 矩阵反复写入 HBM，大幅减少 HBM↔SRAM 的 IO。

第八章 长上下文、高性能 Attention 与 MoE

理解 FlashAttention 的 IO 本质、Sparse/Linear Attention、MoE、量化与长上下文瓶颈。

题号	题目	难度
Q081	FlashAttention 为什么不需要存完整 Attention Matrix?	4/5
Q082	Online Softmax 如何分块仍保持精确?	5/5
Q083	Sparse Attention 如何降低复杂度?	4/5
Q084	Linear Attention 与 FlashAttention 最大区别?	3/5
Q085	长上下文 Transformer 至少有哪些瓶颈?	4/5
Q086	Sliding Window Attention 的优势与损失?	3/5
Q087	MoE 为什么能在相近每-token FLOPs 下增加参数量?	3/5
Q088	MoE 最大训练难点是什么?	4/5
Q089	Quantization 为什么对 Transformer 推理重要?	4/5
Q090	Coding: 从零实现 Multi-Head Attention	3/5

Q081 FlashAttention 为什么不需要存完整 Attention Matrix?

难度 4/5 | 高频 | 面试形式: 概念 / 推导 / 追问

面试官在考什么 理解 block-wise exact computation。

30 秒答案

它按 Q/K/V blocks 计算局部 score，维护每行 Softmax 的 running max、normalization sum 与 output accumulator；当新 block 到来时重缩放旧累积量，从而无需 materialize 全部 $T \times T$ 矩阵。

深度拆解

- 关键是 Softmax 可通过 log-sum-exp state 增量合并。
- 中间结果主要留在片上 SRAM/register。
- backward 也通过重计算/统计避免保存完整概率矩阵。

常见失分点 认为“不存矩阵”意味着不计算这些 pair。

继续追问 相比 activation checkpointing，这种“重算换显存”的思想有何相似/不同？

一语中的：它按 Q/K/V blocks 计算局部 score，维护每行 Softmax 的 running max、normalization sum 与 output accumulator；当新 block 到来时重缩放旧累积量，从而无需 materialize 全部 $T \times T$ 矩阵。

Q082 Online Softmax 如何分块仍保持精确?

难度 5/5 | 高频 | 面试形式: 概念 / 推导 / 追问

面试官在考什么 考察数值算法能力。

30 秒答案

每处理一块 logits, 更新全局 $\max m = \max(m_{old}, m_{new})$, 并把旧的 exp-sum 与 output accumulator 乘 $\exp(m_{old} - m)$ 重新缩放, 再加入新块贡献。这样最终结果与一次性 Softmax 数学等价。

$$l \leftarrow e^{(m_{old} - m)} l_{old} + e^{(m_{new} - m)} l_{new}$$

深度拆解

- 最大值变化时必须重缩放旧贡献。
- 若只把各块独立 softmax 再拼接, 会得到错误结果。
- 需要同时维护 denominator 和加权 V accumulator。

常见失分点 对每个 block 独立 softmax 后直接 concat。

继续追问 请推导两个 block 合并时 denominator 的更新公式。

一语中的: 每处理一块 logits, 更新全局 $\max m = \max(m_{old}, m_{new})$, 并把旧的 exp-sum 与 output accumulator 乘 $\exp(m_{old} - m)$ 重新缩放, 再加入新块贡献。

Q083 Sparse Attention 如何降低复杂度?

难度 4/5 | 高频 | 面试形式: 概念 / 推导 / 追问

面试官在考什么 从连接图稀疏化理解。

30 秒答案

标准 Attention 连接所有 T^2 token pair; Sparse Attention 只保留局部窗口、global token 或特定 block pattern, 使有效边数下降到如 $O(Tw)$ 。代价是某些长距离关系无法直接一层访问。

深度拆解

- 多层局部 attention 可逐层扩大 receptive field。
- 稀疏 pattern 必须匹配任务结构。
- kernel 是否真正高效取决于规则稀疏还是不规则稀疏。

常见失分点 看到 FLOPs 降低就假设 GPU 一定更快。

继续追问 block-sparse 为什么通常比任意 element-wise sparsity 更容易加速?

一语中的: 标准 Attention 连接所有 T^2 token pair; Sparse Attention 只保留局部窗口、global token 或特定 block pattern, 使有效边数下降到如 $O(Tw)$ 。

Q084 Linear Attention 与 FlashAttention 最大区别?

难度 3/5 | 高频 | 面试形式: 概念 / 推导 / 追问

面试官在考什么 区分算法近似/重构与系统实现优化。

30 秒答案

Linear Attention 改变标准 Softmax Attention 的计算形式/函数族以减少随 T 的复杂度；FlashAttention 保留 exact Softmax Attention，只改变 tiling、memory schedule 与 kernel。

深度拆解

- Linear attention 关注 asymptotic complexity。
- FlashAttention 关注 GPU IO complexity。
- 两者可以在不同模型架构中独立讨论。

常见失分点 把它们都归类为“高效 Attention”后不区分是否 exact。

继续追问 为什么在中等长度上 FlashAttention 可能优于理论 $O(T)$ 方法？

一语中的： Linear Attention 改变标准 Softmax Attention 的计算形式/函数族以减少随 T 的复杂度；FlashAttention 保留 exact Softmax Attention，只改变 tiling、memory schedule 与 kernel。

Q085 长上下文 Transformer 至少有哪些瓶颈？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 系统性思维题。

30 秒答案

至少包括 Attention FLOPs、attention/activation memory、KV Cache、position extrapolation、TTFT、长数据稀缺、远距离信息利用能力、分布式通信与评测困难。

深度拆解

- “上下文长度”同时牵涉训练与 serving。
- 长 prompt 的 prefill 通常显著提高 TTFT。
- 即使模型允许 1M token，若有效利用只在近邻，能力价值有限。

常见失分点 只回答 $O(T^2)$ 。

继续追问 哪几个瓶颈可由 FlashAttention 解决，哪些完全不能？

一语中的： 至少包括 Attention FLOPs、attention/activation memory、KV Cache、position extrapolation、TTFT、长数据稀缺、远距离信息利用能力、分布式通信与评测困难。

Q086 Sliding Window Attention 的优势与损失？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解固定窗口让计算/缓存线性化的代价。

30 秒答案

每个 token 只关注前后 w 范围，复杂度与 KV 保留可降至 $O(Tw)/O(w)$ 级别；但跨越窗口的关系需要多层传播或额外 global/retrieval 机制。

深度拆解

- 特别适合局部依赖强的长序列。
- 窗口大小影响质量、延迟和 cache memory。
- 分层/混合 global attention 可补足远距离。

常见失分点 说“多层后就等价全 Attention”；信息路径与容量仍不同。

继续追问 L 层、window w 时理论 receptive field 如何增长？

一语中的：每个 token 只关注前后 w 范围，复杂度与 KV 保留可降至 $O(Tw)/O(w)$ 级别；但跨越窗口的关系需要多层传播或额外 global/retrieval 机制。

Q087 MoE 为什么能在相近每-token FLOPs 下增加参数量？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 理解稀疏激活。

30 秒答案

MoE 有很多 experts，但 router 每个 token 只选择 top-k experts，因此总参数可很大，而单 token 只激活少数 FFN 参数。它提高的是总容量，不等于每 token 都用全部参数。

深度拆解

- 常见 MoE 替换 FFN，而 Attention 仍 dense。
- top-1/top-2 路由影响质量和通信。
- 总参数、激活参数、训练 FLOPs 必须分别报告。

常见失分点 把“1T 参数 MoE”直接当成“每 token 计算 1T 参数”。

继续追问 为什么 MoE 模型推理仍可能需要很多 GPU，即使激活参数不多？

一语中的：MoE 有很多 experts，但 router 每个 token 只选择 top-k experts，因此总参数可很大，而单 token 只激活少数 FFN 参数。

Q088 MoE 最大训练难点是什么？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察通信与负载均衡。

30 秒答案

关键难点包括 router load balance、expert capacity/overflow、跨 GPU All-to-All 通信、router instability 和 expert specialization。算力省下来后，网络通信可能成为主要瓶颈。

深度拆解

- 热点 expert 会造成 straggler。
- auxiliary load-balancing loss 常被使用。
- expert parallel 的拓扑设计非常重要。

常见失分点 只回答“路由器可能选错 expert”。

继续追问 如果 90% token 被路由到一个 expert，会对吞吐和模型训练分别造成什么问题？

一语中的：关键难点包括 router load balance、expert capacity/overflow、跨 GPU All-to-All 通信、router instability 和 expert specialization。

Q089 Quantization 为什么对 Transformer 推理重要？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 连接模型权重、带宽和 KV cache。

30 秒答案

降低权重/激活/KV 的 bit-width 可以减少显存占用和内存带宽，尤其 decode 常为 memory-bound，因此量化不仅让模型装得下，也可能直接提升 tokens/s。

深度拆解

- weight-only INT8/INT4 与 activation quantization 难度不同。
- outlier channel、calibration 和 kernel support 决定实际效果。
- KV cache quantization 直接影响长上下文并发能力。

常见失分点 只比较模型文件大小，不考虑计算 kernel 和精度损失。

继续追问 为什么 INT4 权重不一定获得理论 4× 的 FP16 加速？

一语中的：降低权重/激活/KV 的 bit-width 可以减少显存占用和内存带宽，尤其 decode 常为 memory-bound，因此量化不仅让模型装得下，也可能直接提升 tokens/s。

Q090 Coding：从零实现 Multi-Head Attention

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 代码与 shape 共同考察，必须正确处理 reshape、mask、softmax、合并 heads。

30 秒答案

实现时把 Q/K/V 投影到 [B,H,T,D_h]，计算 scaled score，在 logits 上应用 mask，沿 key 维 softmax，乘 V，再 transpose/reshape 回 [B,T,d] 并做 W_O。

深度拆解

- 保证 d_model%num_heads==0。
- transpose 后注意 contiguous/view。

- mask dtype 与广播 shape。
- 训练/推理 dropout 行为。

代码 / 伪代码骨架

```
class MHA(nn.Module):
    def forward(self, x, mask=None):
        B,T,D = x.shape
        q = self.q(x).view(B,T,self.h,self.dh).transpose(1,2)
        k = self.k(x).view(B,T,self.h,self.dh).transpose(1,2)
        v = self.v(x).view(B,T,self.h,self.dh).transpose(1,2)
        s = (q @ k.transpose(-2,-1)) / math.sqrt(self.dh)
        if mask is not None:
            s = s.masked_fill(~mask, float('-inf'))
        a = torch.softmax(s, dim=-1)
        y = a @ v
        y = y.transpose(1,2).contiguous().view(B,T,D)
        return self.o(y)
```

常见失分点 依赖不断 print shape 才写出来，而不能先写 shape invariant。

继续追问 如何扩展成 cross-attention 和 GQA?

一语中的：实现时把 Q/K/V 投影到 [B,H,T,D_h]，计算 scaled score，在 logits 上应用 mask，沿 key 维 softmax，乘 V，再 transpose/reshape 回 [B,T,d] 并做 W_O。

第九章 Coding / Debug 真题型

将公式转成正确代码，并通过不变量、对照实验和单元测试快速定位实现错误。

题号	题目	难度
Q091	Coding: 下面的 Attention 有哪些 bug?	3/5
Q092	Coding: 给 Transformer 加 KV Cache	4/5
Q093	给你几百行 Transformer 代码，如何系统 Debug?	4/5
Q094	Coding: 如何验证 causal mask 没有未来泄漏?	3/5
Q095	Coding: 如何验证 KV Cache 实现正确?	4/5
Q096	Coding: 为什么 transpose 后 view 经常报错或 silently 出问题?	3/5
Q097	Coding: 如何处理全 Mask 行导致的 NaN?	4/5
Q098	Coding: 如何做 Tiny Overfit Test?	3/5
Q099	Coding: 如何为 Attention 写 Shape Assertions?	3/5
Q100	System Design: 用 Transformer 设计文本分类系统	3/5

Q091 Coding: 下面的 Attention 有哪些 bug?

难度 3/5 | 高频 | 面试形式: 概念 / 推导 / 追问

面试官在考什么 检测候选人是否能在不运行代码时定位算法错误。

30 秒答案

典型错误包括漏除 $\sqrt{d_h}$ 、Softmax 维度错误、mask 放在 Softmax 之后、mask 方向/broadcast 错，以及 half precision 下 -inf/全 mask 行造成 NaN。

深度拆解

- softmax 通常 dim=-1。
- mask 应作用 logits。
- Q/K transpose 维度必须正确。
- 输出 shape 与 residual hidden size 对齐。

代码 / 伪代码骨架

```
# buggy
scores = q @ k.transpose(-2, -1)
attn = torch.softmax(scores, dim=1)
attn = attn * mask
out = attn @ v
```

常见失分点 只找语法 bug，不找数学 bug。

继续追问 如果 loss 仍能下降，怎样证明 mask 逻辑是错的？

一语中的：典型错误包括漏除 $\sqrt{d_h}$ 、Softmax 维度错误、mask 放在 Softmax 之后、mask 方向/broadcast 错，以及 half precision 下 -inf/全 mask 行造成 NaN。

Q092 Coding：给 Transformer 加 KV Cache

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察对增量 Attention 的数据结构理解。

30 秒答案

API 需要接收 `past_k/past_v`，对新 token 只投影 `k_new/v_new`，并把它们写入 cache；Q 只使用当前 token。生产实现避免每步 `torch.cat` 复制历史，通常预分配或使用 block/paged cache。

深度拆解

- cache shape 常为 `[B,H_kv,T,D_h]`。
- position ids 从 `past length` 继续。
- batch 中不同 sequence length 需配合 cache slot/metadata。
- beam reordering 会重排 cache。

代码 / 伪代码骨架

```
def step(x_new, past_k, past_v):
    q = q_proj(x_new)
    k_new = k_proj(x_new)
    v_new = v_proj(x_new)
    # teaching version; production should avoid repeated full concat
    k = torch.cat([past_k, k_new], dim=-2)
    v = torch.cat([past_v, v_new], dim=-2)
    y = attention(q, k, v)
    return y, (k, v)
```

常见失分点 用 `torch.cat` 每步 $O(T)$ 复制整个 cache，却声称已经是高性能实现。

继续追问 如何让同一 prompt 的多个 beam 共享 prefix KV？

一语中的：API 需要接收 `past_k/past_v`，对新 token 只投影 `k_new/v_new`，并把它们写入 cache；Q 只使用当前 token。

Q093 给你几百行 Transformer 代码，如何系统 Debug?

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 看是否有“从不变量到最小化问题”的方法论。

30 秒答案

不要从第一行肉眼扫。先验证 shape、mask、数值范围、概率和、loss alignment、梯度，再做 tiny overfit。用参考实现/禁用复杂优化逐步二分定位。

深度拆解

- 建立每层 invariant: shape、dtype、finite、std。
- attention 每行概率和应接近 1（考虑 dropout 前）。
- 把 flash/fused kernel 切回朴素 PyTorch 对照。
- 在 16-64 个样本上过拟合测试。
- 固定 seed 并简化为单卡 FP32。

常见失分点 把所有模块同时打开，导致无法定位错误源。

继续追问 如何用“二分层数”判断 bug 位于前半还是后半网络？

一语中的： 不要从第一行肉眼扫。

Q094 Coding：如何验证 causal mask 没有未来泄漏？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 要求设计行为级测试，而不仅看矩阵图。

30 秒答案

构造两条序列，使前 t 个 token 完全相同、t 之后不同。在 eval 模式下，比较两条序列位置 $\leq t$ 的 hidden/logits；若 causal mask 正确，它们应在数值误差范围内一致。

深度拆解

- 关闭 dropout。
- 位置编码与 padding 也保持一致。
- 测试多个 t。
- 若使用 batch mask，测试不同 padding 组合。

代码 / 伪代码骨架

```
x1 = [A, B, C, X, Y]
x2 = [A, B, C, P, Q]
# causal model: outputs at positions 0..2 should match
assert_close(model(x1)[:3], model(x2)[:3])
```

常见失分点 只打印 triangular mask，无法覆盖广播/offset bug。

继续追问 KV cache 下如何做同样的 causal leakage test?

一语中的：构造两条序列，使前 t 个 token 完全相同、 t 之后不同。

Q095 Coding：如何验证 KV Cache 实现正确？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 最关键的是 full forward 与 incremental forward 的等价性。

30 秒答案

对同一输入，路径 A 一次性 full forward 得到各位置 logits；路径 B 逐 token 使用 cache。对应位置 logits 应在浮点容差内一致。随后扩展测试 batch、padding、GQA、RoPE offset、cache reset。

深度拆解

- FP16 fused kernel 可能有小数值差异，应使用合理 tolerance。
- 最常见错误不是 K/V concat，而是 position/RoPE offset。
- 必须测试 cache truncate/reorder 场景。

代码 / 伪代码骨架

```
full = model(ids, use_cache=False).logits
past = None
parts = []
for t in range(ids.size(1)):
    out = model(ids[:, t:t+1], past_key_values=past, use_cache=True)
    parts.append(out.logits)
    past = out.past_key_values
inc = torch.cat(parts, dim=1)
torch.testing.assert_close(full, inc, rtol=1e-3, atol=1e-3)
```

常见失分点 只比较最终 greedy token，无法发现 logits 已漂移但 argmax 恰好一致。

继续追问 为什么比较 logits 比比较生成文本更严格？

一语中的：对同一输入，路径 A 一次性 full forward 得到各位置 logits；路径 B 逐 token 使用 cache。

Q096 Coding：为什么 transpose 后 view 经常报错或 silently 出问题？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察 PyTorch stride/contiguity。

30 秒答案

transpose/permute 只改变 stride 视图，内存布局通常不连续；view 要求能按当前 stride 解释成新 shape，否则报错。常用 contiguous().view(...) 或 reshape(...)。

深度拆解

- contiguous 会产生实际内存拷贝。
- reshape 在可能时返回 view，否则复制。
- 高性能代码应避免不必要 layout conversion。

常见失分点 机械加 contiguous 而不理解为什么；在热路径增加额外 copy。

继续追问 FlashAttention kernel 为什么会对输入 layout/stride 有严格要求？

一语中的： transpose/permute 只改变 stride 视图，内存布局通常不连续；view 要求能按当前 stride 解释成新 shape，否则报错。

Q097 Coding：如何处理全 Mask 行导致的 NaN？

难度 4/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 数值稳定性真实坑点。

30 秒答案

若某 query 的所有 key logits 都为 $-\infty$ ，softmax 会出现 0/0 形式并产生 NaN。应保证每个 query 至少有一个合法 key，或在 padding/query mask 逻辑中对无效 query 输出显式置零/跳过。

深度拆解

- left padding + causal mask 组合可能产生前部全 mask 行。
- 某些 fused SDPA kernels 对 fully-masked row 有特定行为，需测试。
- loss mask 仍需排除无效 query。

常见失分点 把 $-\infty$ 换成 $-1e4$ 就认为彻底解决；这可能产生非零概率且依赖 dtype。

继续追问 如何构造最小测试复现 left-padding 的 fully-masked row？

一语中的： 若某 query 的所有 key logits 都为 $-\infty$ ，softmax 会出现 0/0 形式并产生 NaN。

Q098 Coding：如何做 Tiny Overfit Test？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 看是否懂得用最小实验区分实现错误和泛化问题。

30 秒答案

选极小数据集、关闭数据增强/大 dropout，使用足够容量与较大学习率，让模型几百到几千步内把训练 loss 降到极低。如果做不到，优先怀疑数据管线、loss、mask、optimizer 或模型实现。

深度拆解

- tiny overfit 是单元测试，不是最终训练 recipe。
- 先单卡 FP32 简化变量。
- 记录同一 batch loss 曲线最直观。

常见失分点 在 tiny set 上仍保留强 weight decay/dropout/随机采样，降低诊断力。

继续追问 能 overfit 但真实训练不收敛，下一步该看什么？

一语中的：选极小数据集、关闭数据增强/大 dropout，使用足够容量与较大学习率，让模型几百到几千步内把训练 loss 降到极低。

Q099 Coding：如何为 Attention 写 Shape Assertions？

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 把隐式假设变成早失败的程序不变量。

30 秒答案

在关键位置 assert $d_model\%H==0$ 、Q/K head_dim 相等、K/V sequence length 相等、mask 可广播、output hidden size 与 residual 一致。开发阶段尽早 fail-fast，生产可转为静态 shape/kernel constraints。

深度拆解

- shape assertion 能快速定位 transpose 与 GQA group 错。
- dtype/device 也应检查。
- 动态 batch/sequence 长度不要写死。

常见失分点 只靠错误堆栈发现 matmul shape mismatch，调试效率低。

继续追问 如何让单元测试同时覆盖 $B=1/T=1/H=1$ 等边界情况？

一语中的：在关键位置 assert $d_model\%H==0$ 、Q/K head_dim 相等、K/V sequence length 相等、mask 可广播、output hidden size 与 residual 一致。

Q100 System Design：用 Transformer 设计文本分类系统

难度 3/5 | 高频 | 面试形式：概念 / 推导 / 追问

面试官在考什么 考察从模型到数据、指标、部署完整链路。

30 秒答案

先定义标签与业务代价，再设计 tokenizer+encoder+pooling+classifier；训练处理类别不平衡与校准，评估同时看 precision/recall/F1、calibration、latency 与 drift。

深度拆解

- 长文本可能需要 chunking/hierarchical pooling。
- CLS/mean pooling/attention pooling 要用验证集选择。
- 类别阈值应按业务 cost 调整，不一定固定 0.5。
- 线上监控 input drift 与 confidence drift。

常见失分点 只说 “BERT+Linear+CrossEntropy”。

继续追问 如果正样本只有 0.1%，你会优先优化哪些指标与采样策略？

一语中的：先定义标签与业务代价，再设计 tokenizer+encoder+pooling+classifier；训练处理类别不平衡与校准，评估同时看 precision/recall/F1、calibration、latency 与 drift。

加练案例 10 个 System Design 深挖题（不计入 100 题）

为什么单列 系统设计题更适合作为整章案例，而不是为了编号整齐挤进基础题序列。这里把模型、训练、Serving 与数据工程串联起来。

案例 01 System Design：设计 Transformer 机器翻译系统

考察目标 考察 Encoder-Decoder、训练与解码全链路。

主干答案

Source 经 tokenizer+Encoder，Decoder 用 causal self-attention 和 cross-attention 条件生成；训练 teacher forcing+CE，推理可用 beam search，并处理 length normalization、术语约束与质量评估。

展开

- subword tokenizer 影响稀有词。
- BLEU/COMET 等自动指标应结合人工评估。
- encoder K/V 在 decode 时可缓存。
- 术语表/约束解码用于专业领域。

常见失分点 只讲网络，不讲 decoding 与数据。

继续追问 为什么 decoder-only 也能翻译？什么时候你仍会选 encoder-decoder？

案例 02 System Design：设计高吞吐 LLM Serving

考察目标 综合架构题。

主干答案

请求经过 tokenizer→scheduler→prefill→KV allocation→decode loop→sampling→streaming。核心优化包括 FlashAttention、continuous batching、paged KV、GQA、prefix caching、quantization、speculative decoding，并以 TTFT/TPOT/P99 与 throughput 共同约束。

展开

- 将 prefill 与 decode 的资源特性分开建模。
- 调度要防止长 prompt 阻塞短请求。
- KV memory 是并发 capacity 的核心约束。
- 不同模型并行策略影响 latency 与跨卡通信。

常见失分点 只列优化名词，不给 request 生命周期和指标。

继续追问 如果 P99 TTFT 高但 TPOT 很好，你会先查哪一段？

案例 03 System Design：如何估算一个 LLM 服务能承载多少并发？

考察目标 考察容量规划。

主干答案

先用权重显存+运行时 workspace+KV cache 估显存上限，再用 compute/bandwidth 估吞吐上限，二者取更紧者。KV cache 按每请求 context/output 动态增长，因此应使用长度分布而不是只看平均值。

展开

- 可写 $M_{total} \approx M_{weights} + M_{runtime} + \sum M_{KV}(request)$ 。
- 量化、GQA、paged cache 直接改变 memory capacity。
- 吞吐还受 batch、TP、网络与 SLA 限制。

常见失分点 用 GPU 显存/模型权重大小直接得到并发数。

继续追问 怎样用 P95 context length 做更保守的容量规划？

案例 04 System Design：如何设计长上下文训练方案？

考察目标 把数据、position、attention kernel 与 curriculum 统一考虑。

主干答案

先明确目标长度与有效任务；选择 RoPE scaling/position 方法，使用 FlashAttention/sequence parallel 降低训练成本，构造真实长文与跨段任务，并做长度 curriculum 与长上下文专项评测。

展开

- 只是把 max_position_embeddings 改大通常不够。
- 训练 token budget 会因长 sequence 的二次 attention 成本快速上升。
- packing 与 document boundary 设计很重要。
- 评测需覆盖 retrieval 与 reasoning。

常见失分点 只从 RoPE scaling 角度回答。

继续追问 如果训练只能承受 32K，但希望推理 128K，如何设计 staged extension？

案例 05 System Design：如何训练一个 70B Transformer？

考察目标 考察并行、显存、数据与可靠性。

主干答案

先根据参数/optimizer/activation 估显存与 FLOPs，选择 DP/FSDP、TP、PP、CP 组合；使用 BF16、activation checkpointing、FlashAttention；数据按 token 流式，训练监控 loss、grad norm、throughput、MFU 与故障恢复。

展开

- 70B BF16 权重本身约 140GB，还不含 optimizer。
- Adam optimizer states 可能是权重数倍，需要分片。
- 长 context 会显著增加 activation。
- 集群通信拓扑决定 TP/PP 边界。

常见失分点 回答“用 8 张 A100”但不做任何内存估算。

继续追问 请粗算 AdamW 全精度 optimizer state 的额外内存量级。

案例 06 System Design：如何做 Transformer 预训练数据 Packing?

考察目标 连接训练效率与 loss/mask 正确性。

主干答案

Packing 把多个短文档拼入固定长度 sequence，减少 padding 浪费；但必须处理 document boundary，防止不希望的跨文档 attention 或正确地允许连续 LM，并确保 labels/EOS 与 loss mask 对齐。

展开

- packing policy 会改变数据分布。
- 序列末尾残余空间可用 best-fit/greedy packing。
- 跨文档 attention 是否允许取决于 recipe。
- position ids 可以连续或按文档重置，必须与模型假设一致。

常见失分点 只关注 token utilization，忽略边界泄漏。

继续追问 如果 position id 每文档重置，但 attention 又允许跨文档，会产生什么不一致？

案例 07 综合：现代 Decoder-only Transformer Block 与 2017 原版有何演化？

考察目标 检查你能否把十年演化串成因果链。

主干答案

现代 block 常见 RMSNorm/Pre-Norm、RoPE、GQA、FlashAttention kernel、SwiGLU MLP，服务端配 KV cache/PagedAttention；2017 原版是 Encoder-Decoder、sinusoidal PE、MHA、Post-LN 风格、ReLU FFN。演化主线是训练稳定性、长上下文、参数效率与推理内存/IO。

展开

- 架构变化与系统 kernel 要分层描述。
- FlashAttention 不改变模型函数；GQA 改变 K/V head 结构。

- RoPE 改变位置注入；SwiGLU 改变 FFN。
- Decoder-only 是任务/训练接口变化。

常见失分点 把所有新技术都称为 “Attention 优化”。

继续追问 如果只能选三个改动让 2017 Transformer 更像现代 LLM，你选哪三个并说明理由。

案例 08 综合：从公式到 GPU，为什么 Attention 是 “算法+系统” 共同问题？

考察目标 判断是否具备研究/核心算法岗跨层思维。

主干答案

Attention 的质量来自 Q/K/V 与 Softmax 函数，训练稳定性依赖尺度/Norm/精度，速度依赖矩阵形状、HBM IO、kernel 融合，推理还受 KV cache、batch scheduler 和网络通信影响。仅分析 FLOPs 会遗漏大量实际瓶颈。

展开

- 同一数学 Attention 可由朴素 PyTorch、SDPA、FlashAttention 产生不同 wall-clock。
- 同一模型在 prefill 与 decode 的瓶颈不同。
- 算法改动要同时验证 quality、latency、memory 和 system compatibility。

常见失分点 只用 big-O 判定工程优劣。

继续追问 设计一个实验比较两种 attention 实现时，需要控制哪些变量？

案例 09 综合：面试官让你优化一个 Transformer，你如何先定位瓶颈？

考察目标 考察 profiling-first 的工程习惯。

主干答案

先明确目标是训练吞吐、TTFT、TPOT、显存还是质量；用 profiler 与指标分解确认 compute-bound、memory-bound、communication-bound 还是 data-bound，再针对性选择 FlashAttention、量化、GQA、batching、并行或数据优化。

展开

- 先测后改。
- 训练看 MFU、kernel time、all-reduce、data loader stall。
- serving 看 queue/prefill/decode/KV occupancy。
- 优化必须做 correctness/quality regression。

常见失分点 看到 “慢” 就直接换 FlashAttention。

继续追问 如果 GPU utilization 只有 30%，但 HBM bandwidth 已很高，说明什么？

案例 10 终极：如何回答一个你没见过的 Transformer 变体？

考察目标 这是“举一反三”能力的最终模板。

主干答案

把新模块拆成：它改了信息流哪里？数学函数是什么？shape/复杂度如何？训练稳定性如何？推理/缓存是否变化？硬件友好吗？与 baseline 的质量-成本 trade-off 如何验证？按这六步即使没见过论文，也能构造高质量分析。

展开

- 先画数据流，再写公式。
- 再做 asymptotic 与实际 hardware 两层复杂度。
- 检查数值范围、Norm 与 residual。
- 检查 training/inference 是否一致。
- 最后设计 ablation 与 benchmark，而不是凭直觉下结论。

常见失分点 遇到陌生名词就陷入“我没看过这篇论文”。

继续追问 请用这套框架现场分析“把 Softmax 换成 sigmoid gating 的 attention”会有哪些潜在变化？

附录 A 高频公式速查

主题	公式
Scaled Attention	$Attention(Q,K,V)=softmax(QK^T/\sqrt{d_k+M})V$
Causal LM	$P(x_1:T)=\prod_t P(x_t x_{<t})$
LayerNorm	$LN(x)=\gamma(x-\mu)/\sqrt{(\sigma^2+\epsilon)}+\beta$
RMSNorm	$RMSNorm(x)=\gamma x/\sqrt{(\text{mean}(x^2)+\epsilon)}$
Pre-Norm	$x_{l+1}=x_l+F(\text{Norm}(x_l))$
Residual gradient	$\partial (x+F(x))/\partial x=I+\partial F/\partial x$
Attention cost	$O(Td^2+T^2d)$
KV Cache	$M=2\cdot L\cdot B\cdot T\cdot H_{kv}\cdot D_h\cdot \text{bytes}$
RoPE	$(R_m q)^T(R_n k)=q^T R_{\{n-m\}}k$
ALiBi	$\text{score}_{ij}=q_i^T k_j - m_h i-j $
Position interpolation	$p'=p\cdot T_{\text{train}}/T_{\text{new}}$
SwiGLU	$[SiLU(xW_1)\odot(xW_2)]W_3$
Gradient clip	$g\leftarrow g\cdot \min(1,c/\ g\)$
AdamW decay	$\theta\leftarrow(1-\eta\lambda)\theta-\eta\cdot \text{AdamUpdate}$

附录 B 30 个实现级 Gotchas

- 101. Softmax 维度写错：通常沿 key 维 dim=-1。
- 102. Mask 放在 Softmax 之后。
- 103. Causal mask 上/下三角方向反。
- 104. Padding mask 与 loss mask 混为一谈。
- 105. left padding 时 position id 未按有效 token 计算。
- 106. 整行全 -∞ 导致 softmax NaN。
- 107. 漏除 $\sqrt{d_{\text{head}}}$ 。
- 108. transpose 后直接 view，stride 不连续。
- 109. 把 [B,T,H,Dh] 与 [B,H,T,Dh] 混用。
- 110. GQA 中 Q head 与 KV head group 映射错误。
- 111. RoPE 在 K cache 追加时 position offset 错。
- 112. KV cache 每步 torch.cat 导致 $O(T^2)$ 数据搬运。
- 113. cache reset/reorder 在 batch/beam 场景漏处理。
- 114. full forward 与 incremental logits 未做等价性测试。
- 115. 验证时忘记 model.eval()。
- 116. AMP gradient clipping 前未 unscale。
- 117. gradient accumulation 忘记 loss/K。
- 118. scheduler 按 micro-step 而非 optimizer-step 误更新。
- 119. labels 重复 shift 或完全没 shift。
- 120. pad/eos 共用 id 时错误忽略所有 eos loss。
- 121. packed sequence 未处理 sample boundary。
- 122. fused kernel 与朴素实现 mask 语义不一致。
- 123. FlashAttention 被误认为降低数学 $O(T^2)$ FLOPs。
- 124. KV cache 内存估算忘记 layers 或 K/V 两份。
- 125. Beam Search KV 内存忘记乘 beam size。

- 126. 只看平均 latency，不看 P99。
- 127. 静态 batch 被最长请求拖住。
- 128. 量化只看 bit 数，不检查 kernel 是否真支持。
- 129. 多 GPU 只看显存，不看 all-reduce/all-to-all 通信。
- 130. Tiny overfit 失败时仍继续做大规模训练。

附录 C 面试回答模板：六层法

1. 定义 先用一句话给出模块/问题的精确定义。

2. 公式与 Shape 写核心公式，明确输入输出 tensor shape。

3. Why 说明它解决旧方案的什么瓶颈，避免只背结论。

4. 复杂度 分别分析参数量、FLOPs、显存、带宽和通信。

5. Gotcha 主动指出最常见实现坑/数值问题。

6. 追问与 Trade-off 讨论在训练/推理/长上下文/不同硬件下何时不适用。

附录 D 30 天刷题路径

第 1-5 天 Q001-Q025：架构 + Attention。目标：公式、shape、mask 能脱稿。

第 6-9 天 Q026-Q045：Position + Norm + FFN。目标：RoPE、Pre-LN、SwiGLU 能推导。

第 10-13 天 Q046-Q067：BERT/GPT + 训练。目标：目标函数、NaN、优化器、多卡。

第 14-18 天 Q068-Q090：KV Cache + Serving + FlashAttention/MoE。目标：能估内存与 latency。

第 19-23 天 Q091-Q100：全部 Coding/System Design 实做。

第 24-27 天 随机抽 30 题，每题 30 秒 + 3 分钟 + 3 层追问。

第 28-30 天 模拟面试：每场 45-60 分钟，复盘所有“答不出 why”的点。

考文一步

- Vaswani et al. (2017). Attention Is All You Need. NeurIPS. arXiv:1706.03762.
- Devlin et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.
- Liu et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.
- Xiong et al. (2020). On Layer Normalization in the Transformer Architecture. arXiv:2002.04745.
- Shazeer (2020). GLU Variants Improve Transformer. arXiv:2002.05202.
- Su et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864.
- Fedus et al. (2021). Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. arXiv:2101.03961.
- Hoffmann et al. (2022). Training Compute-Optimal Large Language Models. arXiv:2203.15556.
- Dao et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135.
- Leviathan et al. (2022). Fast Inference from Transformers via Speculative Decoding. arXiv:2211.17192.
- Ainslie et al. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. arXiv:2305.13245.
- Kwon et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180.

版权说明 本手册为针对 Transformer/NLP/LLM 算法面试重新编写的原创学习材料。仅借鉴《剑指 Offer》“以题促学、逐层分析、举一反三”的教育方法，不复制其正文、习题原文或特有版式。